2026-08-14 15:17:40 You

2026 07 08 OneToOne Monthly Review(2)...
15.9 KB

analiza el archivo adjunto paso a paso

ChatGPT gpt-5.6-sol-wm

Voy a revisar la transcripción completa para identificar su duración, participantes y temas, dividirla en partes equilibradas y presentarte ahora únicamente la primera parte con sus marcas de tiempo.

La transcripción dura aproximadamente 19 minutos y 37 segundos. La dividiré en dos partes equilibradas:

- Parte 1: 00:02–09:12
- Parte 2: 09:12–19:37

Parte 1 — Revisión de objetivos y fortalecimiento del ownership

00:02 – Contexto de la revisión mensual

Amit solicita a Jhon un resumen del trabajo realizado durante los últimos meses y le pide recordar cuál era su objetivo anual. La intención es revisar el progreso alcanzado y definir cómo debería evolucionar dicho objetivo.

00:22 – Conocimiento de los productos de PaymentEvolution

Jhon explica que su trabajo reciente estuvo orientado a comprender mejor los productos y funcionalidades de PaymentEvolution.

Como parte de este proceso:

- Mantuvo reuniones con Andrea.
- Recibió explicaciones sobre módulos y funcionalidades de Payroll.
- Revisó, entre otros temas, el módulo de empleados y el nuevo Payroll.
- Creó documentos con la información recopilada.
- Compartió esos documentos con Andrea.
- Planea continuar profundizando su conocimiento sobre los productos.

Amit confirma que estas actividades buscan que Jhon desarrolle una comprensión más sólida del dominio de Payroll.

02:31 – Objetivo anual original

Jhon presenta los componentes principales de su objetivo anual:

- Recopilar más información sobre PaymentEvolution y sus iniciativas.
- Profundizar su conocimiento del producto y del dominio de los clientes.
- Incrementar la colaboración con los stakeholders.
- Utilizar inteligencia artificial para mejorar la claridad de los requisitos.

Jhon añade que, después de sus conversaciones recientes con Amit, quiere ajustar el objetivo para fortalecer su ownership. En particular, desea comprender mejor qué sucede después de finalizar el desarrollo de una funcionalidad y cómo se garantiza que llegue efectivamente a producción.

Reconoce que todavía necesita entender mejor:

- Las etapas posteriores al desarrollo.
- El proceso de entrega.
- Los pasos requeridos para mover una funcionalidad a producción.
- El seguimiento necesario hasta completar el ciclo de vida de una iniciativa.

04:12 – Acceso al documento de objetivos

Amit solicita acceso de edición al archivo utilizado durante la reunión. Aunque Jhon actualiza los permisos, Amit continúa viendo dificultades para editarlo.

Finalmente acuerdan que:

- Jhon terminará primero de utilizar y cerrar el archivo.
- Amit agregará posteriormente sus comentarios.
- Amit notificará a Jhon una vez que haya actualizado el documento.

06:31 – Recomendación para concretar el objetivo

Amit considera que el objetivo general es positivo, pero demasiado amplio. Señala que referirse únicamente a dos iniciativas no proporciona una visión suficiente, dado el número de productos y módulos que Jhon debe administrar.

Recomienda comenzar con una estructura sencilla y práctica: crear un documento centralizado que contenga todos los productos, proyectos, módulos e iniciativas bajo seguimiento.

El documento puede administrarse en una herramienta como:

- Loop.
- Excel.
- Jira Product Discovery (JPD).

La herramienta específica no es lo más importante; lo fundamental es disponer de una fuente centralizada y actualizada.

06:39 – Inventario de productos, módulos e iniciativas

Amit explica que Jhon debería registrar claramente las iniciativas de cada área. Por ejemplo:

- Las iniciativas activas dentro de Dashboard.
- Los módulos o reportes que forman parte de Reports.
- Los componentes que se están trabajando dentro de AdminAPI.

Para cada producto, módulo o iniciativa se debería identificar como mínimo:

- Estado actual.
- Nivel de prioridad.
- Trabajo pendiente.
- Próximo paso.
- Necesidad de seguimiento.

Una vez recopilada esa información, Jhon podría reconocer cuáles iniciativas requieren atención inmediata y organizar mejor su trabajo.

07:41 – Payrun History como ejemplo práctico

Amit utiliza Payrun History para ilustrar el proceso.

Si Jhon identifica que:

- La funcionalidad fue desarrollada hace tiempo.
- No conoce claramente su estado actual.
- No sabe si fue desplegada.
- No ha recibido retroalimentación.
- No sabe si requiere seguimiento.

Entonces debería clasificarla como una iniciativa de alta prioridad hasta confirmar su situación.

El objetivo no es realizar personalmente todo el trabajo, sino identificar los vacíos de información, establecer prioridades y promover las acciones necesarias para que la iniciativa avance o pueda cerrarse formalmente.

08:18 – Organización progresiva del trabajo

Amit recomienda que Jhon empiece con una lista manejable, incluso si inicialmente no incluye todos los proyectos.

El proceso propuesto sería:

1. Documentar las iniciativas conocidas.
2. Registrar su estado actual.
3. Asignarles una prioridad.
4. Atender primero las de alta prioridad.
5. Completar sus seguimientos.
6. Continuar progresivamente con los siguientes elementos.

La construcción inicial del inventario podría tomar solamente una o dos horas. Después, el documento serviría para organizar el trabajo durante las semanas siguientes.

08:46 – Rol de Jhon como facilitador y orquestador

Amit aclara que el rol esperado de Jhon no consiste necesariamente en ejecutar todas las soluciones. Como Business Analyst, debe actuar como facilitador y orquestador.

Esto implica:

- Detectar cuándo existe un problema o bloqueo.
- Identificar qué integrante del equipo puede resolverlo.
- Promover la creación de los tickets correspondientes.
- Coordinar las acciones necesarias.
- Dar seguimiento a los responsables.
- Verificar que las iniciativas avancen hasta su implementación.

El punto central es que Jhon tenga visibilidad de cada producto, comprenda su situación y movilice al equipo cuando se requiera una acción.

¿Deseas que continúe con la parte 2, desde 09:12 hasta 19:37?

2026-08-14 15:22:32 **You**

parte 2

ChatGPT gpt-5.6-sol-wm

Parte 2 — Seguimiento de iniciativas, medición del éxito y nuevo Paywall

09:12 – El Business Analyst como orquestador

Amit profundiza en el concepto de “orquestador”. Explica que, cuando Jhon detecte un problema, no necesariamente debe resolverlo directamente; debe identificar quién puede atenderlo y coordinar las

acciones para que se complete.

Entre sus responsabilidades estarían:

- Identificar problemas o necesidades.
- Priorizar los tickets correspondientes.
- Asignar o coordinar el trabajo con el integrante adecuado.
- Verificar que el desarrollo termine.
- Confirmar que el cambio se despliegue.
- Dar seguimiento después de su llegada a producción.

El éxito no se limita a que el desarrollo esté terminado: la funcionalidad debe desplegarse y comprobarse que resolvió el problema original.

09:48 – Ejemplo de seguimiento a un problema de invoicing

Amit utiliza como ejemplo una corrección de invoicing que Brad habría completado, pero que todavía no se había desplegado.

En una situación similar, Jhon debería:

1. Registrar que el desarrollo está terminado pero pendiente de despliegue.
2. Consultar cuándo se realizará el despliegue.
3. Crear un recordatorio para verificarlo en la fecha acordada.
4. Recordar al responsable antes de que venza el plazo.
5. Confirmar que el cambio llegó a producción.
6. Programar una validación posterior para comprobar el resultado.

Amit menciona el viernes como posible fecha de despliegue. Jhon podría configurar un recordatorio para el jueves, consultar nuevamente el viernes y cerrar esa etapa únicamente después de confirmar el despliegue.

10:28 – Validación posterior a producción

El seguimiento debe continuar después del despliegue. Si el problema sucede mensualmente y podría reaparecer el 5 de agosto, Jhon debería registrar esa fecha y volver a consultar al stakeholder.

La validación consistiría en confirmar:

- Si volvieron a presentarse las facturas con errores.
- Si la corrección funciona correctamente.
- Si el stakeholder está satisfecho.
- Si todavía existe algún problema pendiente.

Solo después de obtener una respuesta positiva se debería marcar la iniciativa como cerrada.

11:03 – Documento central como fuente de seguimiento

Amit indica que el documento, independientemente de si se implementa en Excel, Loop o JPD, debe convertirse en la principal fuente de control para Jhon.

Debería mostrar claramente:

- El estado de cada producto o iniciativa.
- La acción pendiente.
- El responsable.
- La prioridad.
- La próxima fecha de seguimiento.
- El resultado de la validación.
- Si la iniciativa puede cerrarse.

Amit lo describe como una especie de “Biblia” o “ticket dorado” para conocer en todo momento dónde se encuentra cada iniciativa. Considera que el objetivo anual debería centrarse precisamente en organizar el trabajo, mantener actualizados los estados y utilizar esa información para priorizar, impulsar y orquestar acciones.

11:49 – Alineación sobre el nuevo objetivo

Jhon confirma que su intención era similar, aunque reconoce que Amit ha llevado la propuesta a un nivel mucho más concreto y profundo.

Amit celebra que ambos estén alineados. Explica que resulta difícil administrar aproximadamente quince proyectos confiando únicamente en la memoria. Un inventario central permitiría conocer qué permanece activo y qué puede eliminarse de la lista de pendientes.

12:13 – Criterios para cerrar una iniciativa

Payrun History vuelve a utilizarse como ejemplo. Una iniciativa podría considerarse cerrada cuando:

- Fue desplegada.
- Está disponible para todos los clientes correspondientes.
- No se han reportado errores durante un periodo razonable, como tres meses.
- No quedan seguimientos pendientes.

Una vez cerrada, Jhon podría dejar de dedicarle atención operativa y evaluar, cuando corresponda, posibles mejoras para una versión 2 o 3.

El documento no solo ayudaría a cerrar el trabajo actual, sino también a identificar la evolución futura de cada producto.

12:57 – Reuniones periódicas con Amit

Amit resume su recomendación: el objetivo debe enfocarse en crear y mantener actualizado el documento de seguimiento.

Propone realizar reuniones breves de aproximadamente diez minutos:

- Semanalmente, o
- Cada dos semanas.

Estas reuniones servirían para revisar si:

- Falta alguna iniciativa.
- El estado registrado es correcto.
- Amit conoce información diferente.
- Las prioridades necesitan ajustarse.
- Existe algún bloqueo que Jhon deba gestionar.

13:31 – Producción como una “caja negra”

Jhon comenta que anteriormente percibía el proceso de despliegue a producción como una “caja negra” o algo reservado. Recuerda haber consultado tiempo atrás si Payrun History había sido desplegado sin recibir una respuesta clara.

Considera que las nuevas reuniones periódicas le permitirán conocer mejor qué está sucediendo con los productos y hacer un seguimiento más efectivo.

14:22 – Transparencia y restricciones de seguridad

Amit reconoce que pudo haber generado esa percepción al no responder algunos mensajes debido a la cantidad de responsabilidades que maneja. Aclara que no tiene intención de ocultar información y se compromete a ser transparente con Jhon.

Sin embargo, explica que los despliegues a producción sí tienen restricciones especiales:

- Involucran datos de clientes.
- Existen lineamientos de seguridad.
- Solo un grupo limitado de personas puede participar directamente.
- No toda la información operativa puede compartirse libremente.

Aun así, Amit indica que, cuando no pueda proporcionar cierta información, se lo comunicará expresamente a Jhon en lugar de dejar la pregunta sin respuesta. También le pide que continúe haciendo seguimiento y formulando las preguntas necesarias.

15:46 – Cómo medir el éxito del objetivo anterior

Amit pregunta cómo se midió el éxito del objetivo relacionado con investigar los productos y aprender sobre Payroll.

Jhon responde que creó documentación con la información recopilada y considera que puede resultar útil para incorporar a nuevos integrantes. Menciona que documentos similares fueron valiosos cuando Mario se unió al equipo.

Por lo tanto, el valor del trabajo no está únicamente en haber adquirido conocimiento, sino en haberlo convertido en material reutilizable por otras personas.

16:46 – Validación de la documentación por Andrea

Amit pregunta si Andrea ya revisó y aprobó la documentación. Jhon confirma que todavía no.

Amit recomienda que Jhon:

- Solicite a Andrea revisar los documentos.
- Confirme que la información sea correcta.
- Realice las correcciones necesarias.
- Obtenga su aprobación antes de utilizar el material como referencia.

Esta revisión funcionaría como una medida objetiva de calidad.

17:01 – Documentación para Igor y Manolo

Amit propone utilizar la documentación con los dos nuevos integrantes, Igor y Manolo.

Esto permitiría validar si:

- El contenido es comprensible.
- La documentación facilita el onboarding.
- Los nuevos integrantes pueden aprovecharla.
- El conocimiento recopilado se convierte en un recurso útil para el equipo.

Amit considera que la revisión de Andrea y el uso efectivo por parte de Igor y Manolo serían buenas evidencias para demostrar el éxito del objetivo anterior.

17:25 – Comentarios en el archivo

Amit confirma que agregará sus observaciones al archivo después de que Jhon lo cierre. Con esto concluye la revisión principal del objetivo anual.

17:40 – Consulta sobre el nuevo Paywall de Reports

Jhon plantea una pregunta adicional relacionada con el proyecto Reports: si se debería utilizar el nuevo Paywall o mantener la implementación clásica.

Amit responde que preferiría construir el nuevo Paywall porque el anterior deberá eliminarse. Sin embargo, condiciona esta decisión a la disponibilidad real de recursos y del equipo necesario para desarrollarlo.

18:24 – Necesidades de Backend, Frontend y UX

Amit identifica las necesidades por área:

- Backend: Kavitha sería adecuada porque conoce el código clásico y podría convertirlo o moverlo hacia servicios.
- Frontend/UX: se necesita una persona con capacidad para implementar la experiencia y el diseño del nuevo Paywall.
- Kruger podría atender la parte de Frontend/UX, siempre que tenga capacidad disponible.

La principal limitación identificada no parece estar en Backend, sino en disponer de un recurso de Frontend con suficiente disponibilidad.

18:52 – Riesgo por sobreasignación de Kruger

Amit advierte que Kruger participa en demasiados proyectos y que trabajar simultáneamente en cinco iniciativas está afectando su capacidad de completarlas exitosamente.

Recomienda que, antes de incluir el nuevo Paywall en el próximo sprint, Jhon consulte con Shreya. Debe verificarse:

- La carga actual de Kruger.
- Su participación en B2B.
- Las demás iniciativas asignadas.
- Si realmente dispone de capacidad para Reports.

Amit considera razonable que una persona maneje un proyecto principal y quizás dos iniciativas menores, pero no cinco proyectos simultáneos.

Jhon acepta coordinarlo con Shreya.

19:31 – Presentación con Igor

Antes de finalizar, Amit pide a Jhon que se presente con Igor, uno de los nuevos integrantes. Jhon acepta realizar la presentación.

Con esto concluye la reunión.

¿Deseas que ahora prepare los puntos de acción de cada integrante y, de forma separada, las tareas específicas que deberías realizar como Business Analyst?

2026-08-14 15:33:18 **You**

Action points

Puntos de acción

Jhon — Business Analyst

- Crear un documento centralizado para registrar todos los productos, módulos e iniciativas bajo su responsabilidad.
- Incluir para cada iniciativa:
  - Estado actual.
  - Prioridad.
  - Próximo paso.
  - Responsable.
  - Fecha de seguimiento.
  - Estado del despliegue.
  - Validación posterior a producción.
  - Criterio de cierre.
- Registrar inicialmente las iniciativas de Dashboard, Reports y AdminAPI.
- Revisar especialmente el estado de Payrun History y confirmar:
  - Si fue desplegado en producción.
  - Si está disponible para todos los clientes.
  - Si existen errores pendientes.
  - Si puede cerrarse o requiere seguimiento.
- Priorizar las iniciativas con estado desconocido, despliegues pendientes o falta de retroalimentación.
- Dar seguimiento a las funcionalidades hasta completar todo su ciclo: desarrollo, despliegue, validación y cierre.
- Crear recordatorios de calendario para los compromisos y verificaciones posteriores a producción.
- Proponer reuniones de diez minutos con Amit, semanalmente o cada dos semanas, para revisar estados, prioridades y bloqueos.
- Cerrar el archivo de objetivos para que Amit pueda agregar sus comentarios.
- Solicitar a Andrea que revise y valide la documentación creada sobre Payroll.
- Actualizar la documentación según las observaciones de Andrea.

- Compartir la documentación validada con Igor y Manolo como material de onboarding.
- Evaluar la utilidad de la documentación mediante la retroalimentación de los nuevos integrantes.
- Presentarse con Igor.
- Consultar con Shreya la disponibilidad de Kruger antes de asignarle trabajo relacionado con el nuevo Paywall.
- Confirmar si el equipo cuenta con suficientes recursos de Frontend/UX para desarrollar el nuevo Paywall.
- Coordinar el trabajo potencial del Paywall entre Backend y Frontend, si se confirma la capacidad del equipo.

Amit

- Agregar comentarios al documento de objetivos después de que Jhon cierre el archivo.
- Mantener transparencia respecto al estado de los productos y despliegues.
- Informar claramente cuando determinada información no pueda compartirse por restricciones de seguridad.
- Participar en reuniones periódicas breves con Jhon para:
  - Revisar el inventario de iniciativas.
  - Confirmar sus estados.
  - Identificar información faltante.
  - Ajustar prioridades.
  - Orientar los próximos pasos.
- Proporcionar información sobre los despliegues cuando corresponda.
- Apoyar la definición final del objetivo anual de Jhon.

Andrea

- Revisar la documentación de Payroll preparada por Jhon.
- Confirmar si el contenido es correcto y completo.
- Proporcionar observaciones o correcciones antes de que se utilice como material de onboarding.

Shreya

- Revisar con Jhon la carga de trabajo actual de Kruger.
- Confirmar si tiene capacidad para participar en el nuevo Paywall de Reports.
- Considerar sus compromisos actuales, especialmente el trabajo relacionado con B2B y otros proyectos.
- Evitar que sea asignado simultáneamente a un número excesivo de iniciativas.

Kavitha — Backend, acción condicionada

Si se aprueba el desarrollo del nuevo Paywall:

- Analizar la implementación clásica existente.
- Identificar la lógica que debe reutilizarse, adaptarse o migrarse.
- Mover la funcionalidad necesaria hacia los servicios de Backend.
- Coordinar el contrato de integración con Frontend.

Kruger — Frontend/UX, acción condicionada

Si Shreya confirma su disponibilidad:

- Analizar los requerimientos visuales y funcionales del nuevo Paywall.
- Implementar la experiencia de Frontend/UX.
- Coordinar con Backend la integración con los servicios correspondientes.

Igor y Manolo

- Utilizar la documentación de Payroll como parte de su onboarding.
- Informar si el contenido es claro y suficiente.
- Identificar dudas, información faltante o aspectos que necesiten mayor explicación.

Prioridades recomendadas para Jhon

1. Cerrar el archivo para permitir los comentarios de Amit.
2. Crear la estructura del documento central de seguimiento.
3. Registrar las iniciativas conocidas y asignarles estado y prioridad.
4. Confirmar el estado de Payrun History y otros desarrollos pendientes de producción.
5. Programar las reuniones periódicas con Amit.
6. Solicitar la validación de la documentación a Andrea.
7. Compartir el material aprobado con Igor y Manolo.
8. Consultar con Shreya la capacidad de Kruger.
9. Confirmar la viabilidad del nuevo Paywall antes de planificar tickets o comprometerlo para un sprint.

2026-08-14 15:39:16 **You**



	MOJA	TATU	SITA	PRODUCTION
Improvements	OK	Test pending	Test pending	Pending
Payrun History	OK	OK	OK	Confirm with Ameet when it will be available for clients
ePay Report	OK	OK	OK	Confirm with Ameet when it will be available for clients
ePay Report - Improvements	OK	OK	X	X
Summary Report	OK	Testing	X	X
Cheques & Payslips - Transfer	OK	Code Merging	X	X
Cheques & Payslips Report	OK	X	X	X
PayExplorer	Development	X	X	X
Homepage Reports	Development	X	X	X

Quiero actuar sobre este tema: """Crear un documento centralizado para registrar todos los productos, módulos e iniciativas bajo mi responsabilidad.""". He tenido un par de reuniones con Ximena la Tech Lead de Reports para elaborar una matriz de los temas que se han desarrollado como parte de Reportes. Adjunto la matriz que creamos. Posteriormente te pasaré la transcripción de las reuniones. Luego te pediré tus recomendaciones para crear algún metodo para mantener control de los releases.

ChatGPT gpt-5.6-sol-wm

Voy a tomar esta matriz como la línea base del trabajo. Usaré el contexto previo de nuestros análisis para relacionarla con los productos de Reports, pero reservaré las recomendaciones definitivas hasta revisar las transcripciones con Ximena.

Perfecto. Tomaré esta matriz como la línea base del control de releases de Reports.

Actualmente registra nueve líneas de trabajo y su avance por ambiente:

- MOJA: desarrollo.
- TATU: pruebas.
- SITA: preproducción.
- Production: disponibilidad para clientes.

La matriz muestra, entre otros puntos:

- Payrun History y ePay Report llegaron hasta SITA; falta confirmar con Ameet su disponibilidad para clientes.
- Improvements tiene pruebas pendientes en TATU y SITA.
- ePay Report – Improvements llegó hasta TATU.
- Summary Report está en pruebas en TATU.
- Cheques & Payslips – Transfer está en code merging en TATU.
- Cheques & Payslips Report permanece en MOJA.
- PayExplorer y Homepage Reports continúan en desarrollo.

Como fotografía general, es un buen punto de partida. Sin embargo, para convertirla en un mecanismo de control, posteriormente tendremos que aclarar elementos como el significado exacto

de “OK” y “X”, responsables, fechas, versión o alcance del release, tickets asociados, bloqueos, aprobación requerida y siguiente acción.

También consideraré que ya tienes una página Reports Home en Confluence. La solución final debería evitar que termines manteniendo manualmente varias fuentes con información duplicada o contradictoria.

Cuando compartas las transcripciones de las reuniones con Ximena, las analizaré paso a paso para:

1. Reconstruir el significado de cada fila y estado.
2. Identificar decisiones, dependencias y bloqueos.
3. Detectar información que falta en la matriz.
4. Separar desarrollo, promoción, pruebas, aprobación y disponibilidad para clientes.
5. Recomendarte un método sostenible para controlar los releases y realizar seguimientos con Ximena y Ameet.

Puedes enviarme la primera transcripción.

2026-08-14 15:46:22 **You**

2026 08 07 ReportsReleaseStatus.txt
11.4 KB

Aclaración: No tengo acceso a Confluence. X significa que no se ha llegado a ese ambiente. OK que estamos seguros que está en ese ambiente. Adjunto transcripción de la primera reunión

ChatGPT gpt-5.6-sol-wm

Entendido: descarto Confluence como opción disponible. Desde ahora interpretaré “X” como “todavía no alcanzó ese ambiente” y “OK” como “presencia confirmada”. Revisaré la primera reunión completa, la dividiré en partes equilibradas y comenzaré con la parte 1. También usaré el contexto previo únicamente para mantener consistentes los nombres de ambientes y productos.

Gracias por la aclaración. La interpretación oficial de la matriz será:

- OK: presencia confirmada en ese ambiente.
- X: todavía no ha llegado a ese ambiente.
- Los estados intermedios, como Development, Testing o Code Merging, describen la actividad actual.

- Producción técnica y disponibilidad para clientes son estados diferentes.
- Confluence no será considerado como herramienta disponible.

La reunión dura aproximadamente 18 minutos y 27 segundos. La dividiré en dos partes:

- Parte 1: 00:00–09:07
- Parte 2: 09:08–18:27

Parte 1 — Construcción inicial de la matriz

00:00 – Objetivo de la reunión

Jhon propone crear una tabla para organizar las funcionalidades y mejoras desarrolladas recientemente dentro de Reports.

El propósito es reconstruir el estado real de cada iniciativa a través de los ambientes:

- MOJA.
- TATU.
- SITA.
- Producción.

La conversación comienza revisando los principales productos desarrollados y tratando de determinar cuáles deben formar parte de la matriz.

00:42 – Situación inicial de ePay Report

Ximena comenta que ePay Report fue movido a producción, pero todavía no está activado para los clientes.

También indica que lo que se encontraba en SITA habría sido trasladado a producción. Esto introduce una distinción importante para el futuro control:

- El código puede estar desplegado en producción.
- La funcionalidad puede seguir desactivada.
- Los clientes pueden no tener acceso, aunque técnicamente el reporte ya esté en producción.

Por lo tanto, confirmar únicamente el ambiente no será suficiente para determinar si una funcionalidad está realmente disponible.

00:55 – Identificación de los productos de Reports

Jhon y Ximena comienzan a enumerar las iniciativas que deberían aparecer en la matriz:

- Payrun History.
- ePay Report.
- Summary Report.

- Cheques & Payslips.
- PayExplorer.
- Homepage Reports.

Jhon pregunta si falta algún otro producto o iniciativa relacionada con Reports.

02:09 – Incorporación de Homepage Reports

Ximena menciona el componente relacionado con Dashboard. Jhon lo identifica como Homepage Reports y acuerdan incluirlo en la matriz.

Esto confirma que Homepage Reports se considera parte del alcance de seguimiento, aunque esté relacionado con el Dashboard o la página principal.

02:13 – Posible problema de Journal

Jhon pregunta si deberían incluir un ticket relacionado con Journal que Ximena estaba revisando.

Ximena explica que:

- El comportamiento existe desde hace mucho tiempo.
- Todavía no sabe si tiene una solución definitiva.
- Podría evaluarse un ajuste relacionado con el timeout.
- El problema no parece particularmente complejo.

Finalmente deciden no incluirlo en la matriz principal, porque no forma parte de las iniciativas recientes que intentan controlar.

Este punto queda fuera del alcance de la matriz, aunque podría requerir seguimiento técnico independiente.

03:08 – Definición de los ambientes

Jhon propone utilizar las siguientes columnas:

- MOJA.
- TATU.
- SITA.
- Producción.

Pregunta si SITA debe considerarse equivalente a producción o si es necesario mantener una columna adicional.

Ximena explica que, normalmente, después de SITA el código pasa rápidamente a producción, por lo que ambos estados suelen estar muy próximos.

Sin embargo, la conversación posterior demuestra que deben mantenerse separados porque:

- Una funcionalidad puede estar en SITA y todavía no estar en producción.
- Puede estar desplegada en producción, pero no activada.
- Puede estar activada solamente para determinados planes o grupos de clientes.

03:41 – Revisión de Payrun History

Jhon comienza a validar el estado de Payrun History:

- MOJA: presencia confirmada.
- TATU: presencia confirmada.

Ximena confirma estos estados, pero explica que existen mejoras recientes que todavía no han seguido el mismo recorrido.

03:57 – Improvements como línea de trabajo transversal

Ximena menciona un ticket asignado a Alexis que contiene ajustes aplicables a varios reportes.

Entre estas mejoras se encuentra un cambio en la forma de mostrar el año y modificaciones de layout. No se trata solamente de una mejora exclusiva de Payrun History, sino de cambios que podrían afectar varios reportes desarrollados por Alexis.

El equipo detectó la situación a raíz de un problema de producción ocurrido la semana anterior. Durante la revisión descubrieron que:

- Los cambios estaban implementados en MOJA.
- Ricardo ya los había validado en MOJA.
- No habían sido promovidos a TATU.
- Por lo tanto, tampoco estaban en SITA ni en producción.

Esta situación llevó a crear “Improvements” como una línea independiente y transversal dentro de la matriz.

05:00 – Confirmación del alcance de la matriz

Jhon resume provisionalmente las líneas de trabajo que deberían registrarse:

- Improvements.
- Payrun History.
- ePay Report.
- Summary Report.
- PayExplorer.
- Homepage Reports.
- Cheques & Payslips.

Posteriormente, Cheques & Payslips será dividido en Transfer y Report, pero en este punto todavía se maneja como una sola iniciativa.

05:28 – Estado de Improvements

Ximena aclara que las mejoras todavía no han sido movidas desde MOJA.

Alexis conserva el ticket para promover todos los cambios realizados, pero no está confirmado si el trabajo se realizará durante el sprint actual o en el siguiente.

Con la interpretación acordada para la matriz:

- MOJA: OK.
- TATU: todavía no ha llegado; las pruebas quedan pendientes después de la promoción.
- SITA: todavía no ha llegado.
- Producción: todavía no ha llegado.

“Test pending” en TATU no debe interpretarse como que el código ya está allí. En este caso, la promoción a TATU también está pendiente.

05:49 – Necesidad de seguimiento al ticket de Improvements

Jhon recuerda haber visto un mensaje en el que Alexis aparentemente solicitaba mover el trabajo al siguiente sprint, pero no siguió toda la conversación debido al gran volumen de mensajes.

Esto evidencia una dificultad del seguimiento actual: decisiones relevantes sobre promociones y fechas pueden quedar dispersas dentro de conversaciones de chat.

Por el momento, Improvements queda registrado como pendiente de promoción y pruebas.

06:29 – Presencia técnica de Payrun History

Ximena confirma que la versión principal de Payrun History se encuentra en:

- MOJA.
- TATU.
- SITA.
- Producción.

Sin embargo, advierte que las mejoras transversales realizadas posteriormente todavía no están en todos esos ambientes.

Esto significa que el estado del producto base y el estado de sus mejoras deben controlarse por separado.

06:48 – Payrun History no necesariamente disponible

Aunque Payrun History está desplegado en producción, Ximena considera que todavía no está activado para los clientes.

Los nuevos reportes dependen de Plan Design. Mientras el reporte no sea habilitado para determinados planes, no aparece para los usuarios.

Por lo tanto, el estado de Payrun History sería:

- Presencia en producción: confirmada.
- Activación para clientes: no confirmada.
- Grupos o planes habilitados: desconocidos.

07:07 – Activación gradual por planes o grupos

Jhon recuerda que, de acuerdo con Plan Design, las funcionalidades pueden activarse progresivamente para grupos de usuarios.

Ximena confirma esta posibilidad, pero aclara que no tiene acceso directo a esa configuración. Sugiere consultar con Ameet:

- Si Payrun History ya está activado.
- Qué grupos tienen acceso.
- Qué planes lo incluyen.
- Si algún cliente ya lo utiliza.

07:38 – Falta de evidencia sobre el uso en producción

Ximena explica que ha ingresado a algunas cuentas y no ha visto el reporte, pero su acceso es mediante backdoor. Por ello, esta observación no constituye una confirmación definitiva.

También señala que no se ha recibido retroalimentación, comentarios ni reportes de uso. Esto refuerza su impresión de que Payrun History todavía no está disponible para clientes, aunque el código ya se encuentre en producción.

La conclusión provisional es que se necesita una confirmación directa de Ameet.

08:17 – Posible evidencia enviada por Andrea

Jhon recuerda que Andrea le había enviado recientemente una captura relacionada con Payrun History. Inicialmente consideran que podría ser evidencia de que el reporte estaba siendo utilizado en producción.

Ximena pregunta si el mensaje estaba relacionado con un rango de fechas, pero Jhon indica que se trataba de una pantalla de Payrun History.

Al finalizar esta parte, ambos comienzan a revisar la captura para determinar de qué ambiente provenía. La confirmación se produce en la segunda parte.

¿Deseas que continúe con la parte 2, desde 09:08 hasta 18:27?

parte 2

ChatGPT gpt-5.6-sol-wm

Parte 2 — Confirmación de estados y dudas pendientes

09:08 – La captura de Payrun History corresponde a MOJA

Jhon revisa el mensaje enviado por Andrea y confirma que la captura no corresponde a producción, sino a MOJA.

Andrea había señalado que una sección de Payrun History se veía extraña y que posiblemente no mostraba la información correcta. También había incluido comentarios sobre mejoras que le gustaría ver.

Por lo tanto, esta captura:

- No demuestra que Payrun History esté disponible para clientes.
- Corresponde a una validación o comentario realizado en MOJA.
- Podría generar mejoras adicionales para el producto.
- No resuelve la duda sobre su activación en producción.

10:19 – Andrea solicita una fecha estimada

Jhon y Ximena interpretan que Andrea también está preguntando cuándo estará disponible Payrun History para los clientes.

Ximena aclara nuevamente que la responsabilidad técnica del equipo ya habría terminado porque el reporte fue desplegado. Sin embargo, su disponibilidad depende de la activación mediante Plan Design.

La conclusión es que deben confirmar con Ameet:

- Cuándo se habilitará Payrun History.
- Si ya existe algún grupo con acceso.
- Qué planes lo tendrán disponible.
- Si existe una fecha estimada de activación.

La matriz debería distinguir entre:

- Producción: OK, porque el código está presente.
- Disponibilidad para clientes: pendiente de confirmación con Ameet.

10:50 – Revisión de ePay Report

Jhon comienza a validar ePay Report por ambiente:

- MOJA: OK.
- TATU: OK.
- SITA: inicialmente considerado OK.
- Producción: requiere mayor aclaración.

Ximena explica que ePay Report se encuentra en producción, pero aparentemente en una versión anterior y sin usuarios activos.

La situación es similar a Payrun History:

- El código base está presente en producción.
- La versión exacta desplegada no está completamente clara.
- El reporte no parece estar activado para clientes.
- La utilización real no está confirmada.

11:56 – Dudas sobre las mejoras de ePay Report

Ximena diferencia el reporte base de las mejoras posteriores relacionadas con:

- El procesamiento de registros rechazados.
- El flujo de Dynamics.
- Otros ajustes implementados después de la versión inicial.

Inicialmente recuerda que estos cambios podrían haber sido promovidos hasta SITA, pero no tiene certeza.

Esto demuestra que “ePay Report” debe controlarse como mínimo mediante dos líneas:

- ePay Report: versión principal.
- ePay Report – Improvements: cambios posteriores.

Esta separación coincide con la estructura final de la matriz.

12:27 – SITA como “caja negra” para Jhon

Jhon explica que hasta ese momento había controlado las iniciativas principalmente hasta TATU.

Los movimientos posteriores hacia SITA han sido una “caja negra” para él. Sin embargo, como parte de su nuevo objetivo de ownership, necesita comenzar a controlar también:

- La promoción hacia SITA.
- La validación en preproducción.
- El movimiento a producción.

- La activación final para clientes.

Jhon solicita ayuda a Ximena para confirmar el estado de las promociones más recientes.

12:50 – Paola como fuente de confirmación

Ximena no recuerda con certeza si las mejoras recientes de ePay Report llegaron solamente a TATU o también a SITA.

Indica que Paola debería poder confirmar:

- Hasta qué ambiente llegaron los últimos cambios.
- Si las pruebas correspondientes terminaron.
- Si los cambios fueron aprobados.
- Si ya se promovieron a SITA.

Esto identifica a QA como una fuente importante para verificar el último ambiente validado.

13:11 – Consulta pendiente con Paola

Jhon propone llamar a Paola durante la reunión, pero ella no responde.

Se registra como pendiente consultarle directamente. Jhon posteriormente le envía un mensaje para confirmar el estado de ePay Report.

Mientras no exista respuesta, no debería marcarse SITA como confirmado para las mejoras.

13:57 – Posible consulta con Kavitha

Ximena considera consultar también con Kavitha, quien podría conocer el estado técnico de la promoción.

Sin embargo, Kavitha se encontraba terminando trabajo antes de sus vacaciones, por lo que la consulta podía no recibir una respuesta inmediata.

La conversación posterior sobre su viaje no aporta información adicional al control de releases.

15:21 – Dependencia técnica para la promoción

Jhon insiste en que necesitan saber si ePay Report fue aprobado en SITA.

Ximena recuerda que la promoción dependía de que otro componente técnico también estuviera disponible. El nombre de este componente no se distingue con claridad en la transcripción; parece relacionarse con una API o dependencia que Mario habría confirmado como desplegada.

Ximena considera probable que la promoción se haya completado, pero no puede asegurarlo.

Por lo tanto:

- TATU: confirmado.
- SITA: pendiente de confirmación con Paola, Kavitha o evidencia del despliegue.

- Producción: no confirmada para las mejoras.
- Activación para clientes: no confirmada.

La matriz final refleja prudentemente las mejoras de ePay Report con “X” en SITA y producción.

15:47 – Estado de Summary Report

Jhon y Ximena confirman el siguiente estado:

- MOJA: OK.
- TATU: Testing.
- SITA: X.
- Producción: X.

Summary Report ya llegó a TATU, pero todavía se encuentra en pruebas. No debe avanzar a SITA hasta completar satisfactoriamente esa validación.

16:09 – Estado de PayExplorer

PayExplorer continúa en desarrollo y solamente está presente en MOJA.

Su estado queda registrado así:

- MOJA: Development.
- TATU: X.
- SITA: X.
- Producción: X.

Todavía no existe una promoción hacia ambientes de pruebas.

16:20 – Estado de Homepage Reports

Homepage Reports también continúa en desarrollo y solamente está presente en MOJA:

- MOJA: Development.
- TATU: X.
- SITA: X.
- Producción: X.

Al igual que PayExplorer, aún no ha comenzado su recorrido formal por los ambientes posteriores.

16:36 – Cheques & Payslips está más avanzado

Jhon considera que Cheques & Payslips tiene un avance mayor que PayExplorer y Homepage Reports.

Ximena explica que actualmente se trabaja específicamente en promover la parte de transferencia. Este componente parece estar relacionado con una funcionalidad o dependencia previamente

implementada en otra solución, cuyo nombre no queda claramente reconocido por el transcriptor.

Lo importante es que únicamente se está moviendo la parte de Transfer, no el reporte completo.

17:01 – División de Cheques & Payslips

Jhon concluye que Cheques & Payslips debe dividirse en dos líneas independientes:

1. Cheques & Payslips – Transfer.
2. Cheques & Payslips Report.

Esta separación es necesaria porque ambos componentes tienen estados diferentes y no avanzan juntos entre ambientes.

17:17 – Estado de Cheques & Payslips – Transfer

El componente Transfer está confirmado en MOJA y actualmente se realiza el code merge necesario para llevarlo a TATU.

Su estado queda registrado como:

- MOJA: OK.
- TATU: Code Merging.
- SITA: X.
- Producción: X.

“Code Merging” significa que todavía no debe considerarse confirmado en TATU. El trabajo de integración está en curso.

17:41 – Estado de Cheques & Payslips Report

El reporte completo de Cheques & Payslips todavía no ha llegado a TATU.

Su estado es:

- MOJA: OK.
- TATU: X.
- SITA: X.
- Producción: X.

Esto confirma que Transfer y Report requieren seguimiento separado.

18:13 – Cierre de la matriz inicial

Ximena confirma que la tabla representa correctamente el estado conocido durante la reunión.

La matriz queda conformada por nueve líneas:

1. Improvements.

2. Payrun History.
3. ePay Report.
4. ePay Report – Improvements.
5. Summary Report.
6. Cheques & Payslips – Transfer.
7. Cheques & Payslips Report.
8. PayExplorer.
9. Homepage Reports.

Confirmaciones pendientes después de esta reunión

- Consultar con Ameet cuándo Payrun History estará disponible para los clientes.
- Confirmar qué planes o grupos tienen acceso a Payrun History.
- Consultar con Ameet cuándo ePay Report estará disponible para los clientes.
- Confirmar qué versión de ePay Report está desplegada en producción.
- Consultar con Paola hasta qué ambiente llegaron las mejoras de ePay Report.
- Verificar si las mejoras de ePay Report fueron aprobadas y promovidas a SITA.
- Confirmar la dependencia técnica mencionada por Ximena y su estado.
- Confirmar en qué sprint se promoverán las mejoras transversales realizadas por Alexis.
- Dar seguimiento al code merge de Cheques & Payslips – Transfer hacia TATU.

La primera reunión confirma que la matriz es útil, pero también revela cuatro estados diferentes que el futuro método deberá controlar separadamente: presencia en el ambiente, pruebas, aprobación y activación para clientes.

2026-08-14 16:00:42 **You**

2026 08 07 ReportsReleaseStatus 02.txt
7.0 KB

Adjunto ahora transcripción de la segunda reunión, revisemos si encontramos mas información útil para nuestro propósito

ChatGPT gpt-5.6-sol-wm

Voy a contrastar esta segunda reunión con la matriz y con las conclusiones de la primera. Buscaré especialmente nuevos estados, responsables, dependencias, evidencias de promoción y cualquier distinción adicional necesaria para controlar los releases. Mantendré como reglas que “OK” confirma presencia y “X” significa que aún no se llegó al ambiente.

Sí, esta segunda reunión aporta información útil y confirma varios problemas que la matriz actual todavía no representa: existen releases agrupados, versiones diferentes del mismo producto, dependencias técnicas, release notes y activaciones posteriores al despliegue.

Análisis de la segunda reunión

00:00 – Remittances como parte de Improvements

Jhon solicita una explicación sobre el trabajo de Remittances.

Ximena indica que existe una mejora relacionada con Remittances que Kavitha debe incluir en las release notes de Summary Report. Esta mejora podría moverse junto con:

- Summary Report.
- Improvements desarrollados por Alexis.
- Posiblemente ePay Report – Improvements.

Esto revela que varias filas de la matriz pueden formar parte de un mismo paquete de release, aunque representen productos o componentes diferentes.

00:36 – Posible release conjunto

Ximena considera que Summary Report, las mejoras transversales y posiblemente los cambios de ePay podrían promoverse juntos.

Sin embargo, no lo confirma completamente. Por lo tanto, debería registrarse como una hipótesis pendiente de validación:

- Release candidato: Summary Report + Improvements + ePay Report – Improvements.
- Dependencia: confirmar el alcance mediante las release notes.
- Próximo paso: confirmar con Ximena, QA y los desarrolladores qué componentes realmente viajarán juntos.

Esto es importante porque el seguimiento no debería hacerse solamente por producto; también debe existir un identificador para agrupar las iniciativas que pertenecen a una misma promoción.

01:24 – Modernización del proceso de Remittances

Ximena explica el proceso funcional de Remittances:

- Actualmente se generan archivos EDI para procesar remittances.
- La implementación existente utiliza la tabla legacy Payments.

- La lectura o sincronización con Twine genera problemas.
- El proceso actual se ejecuta desde Backoffice/Classic.
- Se seleccionan pagos, se cambian estados y se generan registros para crear el archivo EDI.

El objetivo es modernizar este proceso y trasladarlo progresivamente hacia AdminAPI y nuevas tablas SQL.

02:01 – MRQ como desarrollo nuevo

La generación de archivos EDI para MRQ todavía no había sido desarrollada. Por ello, MRQ será el primer flujo implementado mediante la nueva arquitectura.

La secuencia esperada es:

1. Implementar el nuevo flujo de MRQ en AdminAPI.
2. Obtener la aprobación del archivo generado.
3. Comprobar el funcionamiento del proceso.
4. Utilizar la experiencia para modernizar posteriormente el flujo existente de CRA.
5. Reemplazar gradualmente el proceso Classic.

En la transcripción, “CRI” o “CIA” parecen referirse a CRA, mientras que “DMR-Q” corresponde a MRQ.

02:47 – Migración desde Payments hacia nuevas tablas

La solución propuesta tomaría inicialmente la información de Payments, pero almacenaría los datos necesarios en nuevas tablas SQL utilizadas por la API.

Desde esas nuevas tablas se realizaría el procesamiento:

- Seleccionar los registros que deben enviarse.
- Controlar sus estados.
- Generar el archivo EDI.
- Registrar el resultado del procesamiento.

Esto representa una dependencia técnica y una iniciativa adicional que no aparece claramente en la matriz actual.

03:20 – Aprobación externa del archivo

Ximena se encontraba probando la generación del archivo EDI.

El archivo ya habría recibido una aprobación bancaria, pero todavía se necesitaba la aprobación de la autoridad gubernamental correspondiente.

Por lo tanto, el release no depende únicamente del desarrollo y las pruebas internas; también requiere una validación externa.

El proceso completo de API todavía no estaba terminado. En ese momento solamente se estaba validando la generación del archivo.

04:05 – Personas con conocimiento del proceso

Jhon pregunta quién conoce el proceso de Remittances.

Ximena indica que ahora existe conocimiento distribuido entre varias personas:

- Ameet conoce el objetivo y la modernización esperada.
- Ximena conoce la generación y validación del archivo.
- Felipe trabaja en ajustes del proceso actual de Classic/Backoffice.
- Mario posiblemente conoce parte del flujo técnico.
- Paola realiza validaciones.

El nombre “Niro/Miro” aparece varias veces en la transcripción, pero no se puede identificar con certeza. No debería asignarse una responsabilidad formal basándose únicamente en el texto transcrito.

05:12 – Confusión causada por la clasificación como Payroll

Jhon comenta que desconocía que Felipe y Paola estaban trabajando en Remittances.

Ximena explica que esta iniciativa se comunica como parte de Payroll durante los stand-ups, aunque el trabajo técnico se realiza realmente en:

- Classic/Backoffice.
- AdminAPI.
- El proceso de generación de archivos EDI.

Esto revela un problema de visibilidad: el nombre utilizado en los stand-ups no permite reconocer fácilmente el producto, proceso o release afectado.

05:51 – Estrategia de migración MRQ → CRA

Ximena confirma la estrategia:

- MRQ será implementado primero en AdminAPI porque es un flujo nuevo.
- CRA continuará temporalmente utilizando el proceso existente.
- Cuando el nuevo enfoque esté validado, CRA también se migrará hacia AdminAPI.
- El objetivo final es reemplazar el proceso actual de Classic/Backoffice.

Esta iniciativa tiene fases claramente diferenciadas y no debería registrarse únicamente con un estado general como “Development”.

06:59 – Confirmación de Paola sobre ePay Report

Paola responde durante la reunión y confirma que ePay Report con AdminAPI se encuentra en TATU, no en SITA.

La palabra “dato” que aparece en la transcripción corresponde probablemente a TATU.

Esto aclara la duda de la primera reunión:

- ePay Report base: llegó más lejos y puede estar en SITA o en una versión previa de producción.
- ePay Report – Improvements/AdminAPI: está en TATU.
- ePay Report – Improvements/AdminAPI en SITA: X.
- ePay Report – Improvements/AdminAPI en producción: X.

La separación de ePay Report y ePay Report – Improvements es correcta y necesaria.

07:26 – Necesidad de mantener dos filas de ePay

Ximena confirma expresamente que ePay Report debe mantenerse dividido en dos líneas:

1. ePay Report.
2. ePay Report – Improvements.

El producto base llegó hasta cierto punto en SITA, pero los últimos cambios todavía no. Una única fila ocultaría esta diferencia entre versiones.

07:46 – Release notes como evidencia de alcance

Ximena menciona que las release notes deberían permitir confirmar qué componentes se moverán cuando se promueva Summary Report.

Este es uno de los hallazgos más importantes para el método de control: las release notes pueden actuar como evidencia del contenido de cada release, especialmente porque Jhon no tiene acceso a Confluence.

Sin embargo, para que sean útiles deben:

- Existir antes de la promoción.
- Enumerar todas las funcionalidades y mejoras incluidas.
- Identificar tickets o componentes.
- Especificar el ambiente de origen y destino.
- Actualizarse si el alcance cambia.

08:19 – Falta de seguimiento como problema principal

Jhon y Ximena coinciden en que las iniciativas requieren seguimiento formal. Actualmente pueden ocurrir situaciones como:

- Ameet considera que algo ya fue solicitado.

- El equipo entiende que debería estar terminado.
- Nadie confirma si la acción realmente se ejecutó.
- La funcionalidad queda desplegada pero no activada.
- No existe una persona controlando el siguiente paso.

Jhon identifica que algunos procesos quedan pendientes de una acción de Ameet y que posteriormente no existe seguimiento.

08:43 – Posible activación gradual por empresa

Ximena considera que los reportes podrían activarse inicialmente para empresas específicas, de manera similar a otros lanzamientos graduales.

Sin embargo, no sabe si esa activación ya se realizó ni si alguna empresa utiliza los reportes en producción.

Esto confirma que deben controlarse por separado:

- Despliegue en producción.
- Activación mediante configuración.
- Empresas o planes habilitados.
- Fecha de activación.
- Uso real.
- Retroalimentación posterior.

Información nueva que aporta esta reunión

1. Las iniciativas pueden agruparse en un mismo release.

Summary Report, Improvements y posiblemente ePay Report – Improvements podrían promoverse juntos.

2. Se necesita controlar versiones o alcances.

ePay Report base y sus mejoras están en ambientes diferentes.

3. Las release notes son una fuente clave.

Pueden proporcionar evidencia sobre el contenido exacto de cada promoción.

4. Remittances tiene dependencias externas.

La generación del archivo EDI requiere aprobación de una autoridad externa, además de pruebas técnicas.

5. Remittances tiene varias fases.

Primero MRQ en AdminAPI; posteriormente migración de CRA desde Classic.

6. Los nombres utilizados en los stand-ups dificultan la visibilidad.

El trabajo de Remittances aparece descrito genéricamente como Payroll.

7. Desplegado no significa disponible.

Una funcionalidad puede encontrarse en producción sin estar activada para ninguna empresa.

8. No existe un seguimiento formal después de la solicitud.

Esta es la principal brecha que el futuro método debe resolver.

Impacto sobre la matriz actual

La matriz mantiene correctamente estas filas:

- ePay Report.
- ePay Report – Improvements.

Y confirma para ePay Report – Improvements:

- MOJA: OK.
- TATU: OK.
- SITA: X.
- Producción: X.

No agregaría todavía Remittances como una sola fila sin aclarar su alcance. Antes convendría determinar si se controlará como:

- Una mejora incluida en Summary Report.
- Una iniciativa AdminAPI – MRQ.
- Una migración posterior AdminAPI – CRA.
- Una dependencia técnica del release.

La segunda reunión confirma que la matriz de ambientes es necesaria, pero no suficiente. El método final también deberá controlar paquetes de release, versiones, release notes, aprobaciones, dependencias y activaciones para clientes.

2026-08-14 16:05:56 **You**

Ahora que conoces el contexto completo, Qué me recomiendas hacer? Qué opciones tengo? La matriz original es útil? qué cambios haríamos a la matriz original? cómo controlo estos temas?

Voy a estructurar la recomendación en tres niveles: opciones disponibles, diseño recomendado y cambios concretos sobre tu matriz. Para que el método sea fácil de aplicar, usaré una visualización breve del flujo y separaré claramente “despliegue”, “pruebas”, “aprobación” y “activación”, que fueron las principales brechas detectadas en las reuniones.

Mi recomendación es conservar la matriz, pero convertirla en la vista ejecutiva de un control más completo. Por sí sola es útil como fotografía, pero no permite administrar un release de principio a fin.

La mejor opción para ti ahora es un Excel compartido con información de control y enlaces a Jira. Excel sería tu fuente central para saber dónde está cada iniciativa; Jira conservaría el detalle técnico y las evidencias del trabajo.

¿La matriz original es útil?

Sí. Permite responder rápidamente:

- ¿Qué productos o componentes estamos desarrollando?
- ¿Hasta qué ambiente llegó cada uno?
- ¿Qué elementos están más adelantados?
- ¿Dónde parece haberse detenido una promoción?

Pero actualmente no responde:

- ¿Qué versión o mejoras están en cada ambiente?
- ¿Está desplegado, probado o aprobado?
- ¿Quién confirmó el estado?
- ¿Cuándo fue confirmado?
- ¿Qué tickets forman parte del release?
- ¿Qué iniciativas se moverán juntas?
- ¿Cuál es el siguiente paso?
- ¿Quién debe realizarlo?
- ¿Cuándo debe hacerse el seguimiento?
- ¿Está desplegado en producción pero desactivado?
- ¿Qué clientes o planes tienen acceso?

Por eso no la eliminaría; la evolucionaría.

Opciones disponibles

Opción	Funcionamiento	Ventaja	Limitación
Matriz mejorada	Agregar columnas a la tabla actual	Rápida y sencilla	Pierde historial y se vuelve ancha
Excel estructurado + Jira	Vista general, registro detallado, acciones e historial; Jira conserva el trabajo técnico	Mejor equilibrio entre control y mantenimiento	Requiere disciplina semanal
Jira Releases/Fix Version	Controlar releases, tickets y dashboards directamente en Jira	Mayor trazabilidad y menos duplicación	Puede requerir configuración, permisos y adopción del equipo

Recomiendo comenzar con la segunda opción. Más adelante, cuando el proceso esté estabilizado, se puede evaluar migrarlo parcialmente a Jira Releases.

Estructura recomendada del Excel

1. Overview

Sería la evolución de la matriz original. Mostraría únicamente el estado actual y los temas que requieren atención.

Una fila ya no representaría solamente un producto, sino una unidad de release: un alcance que se mueve junto entre ambientes.

Ejemplos:

- ePay Report – Base.
- ePay Report – AdminAPI/Rejections/Dynamics Improvements.
- Cheques & Payslips – Transfer.
- Cheques & Payslips – Report.
- Reports – Cross-cutting UI Improvements.

2. Release Register

Contendría el detalle necesario para controlar cada unidad:

- Release ID.
- Producto o módulo.
- Alcance o versión.
- Tickets de Jira.
- Release notes.
- Prioridad.

- Responsable de desarrollo.
- Responsable de QA.
- Estado en MOJA.
- Estado en TATU.
- Estado en SITA.
- Despliegue en producción.
- Activación para clientes.
- Planes o empresas habilitadas.
- Dependencias o bloqueos.
- Próxima acción.
- Responsable de la acción.
- Fecha de seguimiento.
- Última verificación.
- Confirmado por.
- Evidencia o enlace.

El Release ID permitirá relacionar elementos que viajarán juntos. Por ejemplo, si Summary Report, Improvements y ePay Improvements forman parte de una misma promoción, compartirían el mismo Release ID.

No deberían agruparse hasta que el alcance sea confirmado mediante las release notes.

3. Follow-ups

Una fila por cada acción pendiente:

Release ID	Acción	Responsable	Fecha objetivo	Estado	Resultado
Por definir	Confirmar activación de Payrun History	Ameet/Jhon seguimiento	Por definir	Pendiente	—
Por definir	Confirmar sprint de Improvements	Alexis/Ximena	Por definir	Pendiente	—
Por definir	Completar pruebas de Summary	QA	Por definir	En progreso	—

Esta pestaña sería tu lista operativa como Business Analyst.

4. Release History

No conviene perder el recorrido cuando se actualice la matriz. Cada cambio importante debería generar un registro:

- Fecha.
- Release ID.
- Producto.
- Ambiente.
- Evento.
- Estado anterior.
- Estado nuevo.
- Confirmado por.
- Evidencia.
- Comentarios.

Ejemplo: “Summary Report fue desplegado en TATU e inició pruebas”.

Cambios concretos en la matriz original

1. Mantener separados SITA y Production

Aunque normalmente el paso sea rápido, representan momentos diferentes y no deben fusionarse.

2. Dividir Production en dos columnas

- Production Deployment.
- Client Activation.

Así se representaría correctamente el caso de Payrun History y ePay Report: código presente, pero disponibilidad para clientes pendiente de confirmación.

3. Reemplazar el “OK” aislado

“OK” confirma presencia, pero no indica si fue probado o aprobado. Recomiendo estos estados:

- X: no ha llegado.
- Development: desarrollo activo.
- Ready to Promote: listo para promoción.
- Code Merging: integración en progreso.
- Deployed: presente en el ambiente; pruebas no confirmadas.
- Testing: pruebas en curso.
- Approved: pruebas terminadas y aprobadas.
- Blocked: no puede avanzar.

- Deployed – Not Activated: producción sin clientes habilitados.
- Pilot: activado para un grupo limitado.
- Active: disponible para el alcance previsto.
- Monitoring: seguimiento posterior al release.
- Closed: release validado y cerrado.

Si deseas mantener “OK”, debería mostrarse como “OK – Deployed” o “OK – Approved”, nunca solo.

4. Agregar “Last verified” y “Confirmed by”

Un estado sin fecha o fuente pierde confiabilidad rápidamente.

Ejemplo:

- Last verified: 2026-08-07.
- Confirmed by: Paola.
- Evidence: mensaje donde confirma que ePay Improvements está en TATU y no en SITA.

5. Agregar “Next action”, “Owner” y “Follow-up date”

Cada fila abierta debe tener exactamente una próxima acción claramente identificada.

No debería existir una fila con “Pending” sin especificar:

- ¿Qué está pendiente?
- ¿Quién debe hacerlo?
- ¿Cuándo se revisará nuevamente?

6. Agregar “Release ID” y “Release notes”

Esto permitirá controlar promociones conjuntas y evitar que un componente quede olvidado en MOJA.

7. Renombrar filas demasiado generales

Fila actual	Nombre recomendado
Improvements	Reports – Cross-cutting UI Improvements
Payrun History	Payrun History – Base Release
ePay Report	ePay Report – Base Release
ePay Report – Improvements	ePay Report – AdminAPI/Rejections/Dynamics Improvements
Cheques & Payslips – Transfer	Se mantiene
Cheques & Payslips Report	Se mantiene

Fila actual	Nombre recomendado
Homepage Reports	Reports Homepage

Si aparecen mejoras específicas de Payrun History que tengan un recorrido diferente, deberán registrarse en una fila adicional.

Vista mejorada usando el estado conocido

Unidad de release	MOJA	TATU	SIT A	Producción	Activación	Próxima acción
Cross-cutting UI Improvements	Tested	X – promoción pendiente	X	X	N/A	Confirmar sprint y promoción
Payrun History – Base	Deployed	Deployed	Deployed	Deployed	Sin confirmar	Consultar fecha, planes y empresas con Ameet
ePay Report – Base	Deployed	Deployed	Deployed	Versión anterior desplegada	Sin confirmar	Confirmar versión y disponibilidad
ePay Report – Improvements	Deployed	Deployed/ Validated	X	X	N/A	Preparar release notes y promoción
Summary Report	Deployed	Testing	X	X	N/A	Completar QA y obtener aprobación
Cheques & Payslips – Transfer	Deployed	Code Merging	X	X	N/A	Terminar merge y confirmar despliegue
Cheques & Payslips – Report	Deployed	X	X	X	N/A	Definir promoción
PayExplorer	Development	X	X	X	N/A	Confirmar alcance y readiness
Reports Homepage	Development	X	X	X	N/A	Confirmar alcance y readiness

Estos estados deben considerarse una línea base del 7 de agosto de 2026, no estados vigentes indefinidamente.

¿Debemos agregar Remittances?

Sí al inventario general, si está bajo tu responsabilidad, pero no como una única fila genérica.

La dividiría inicialmente en:

- Remittances – MRQ EDI/AdminAPI.
- Remittances – CRA Migration to AdminAPI.

La mejora incluida dentro de Summary Report se identificaría en el alcance o las release notes de Summary. No la duplicaría como otro release independiente si se despliega necesariamente junto con Summary.

Antes de agregar estas filas deben confirmarse:

- Alcance exacto.
- Producto o proyecto propietario.
- Tickets relacionados.
- Responsables.
- Dependencias externas.
- Criterio de aprobación gubernamental.

Flujo que deberías controlar

```mermaid
flowchart TD
    A["Alcance y Release ID confirmados"] --> B["MOJA: desarrollo y validación"]
    B --> C["TATU: despliegue, pruebas y aprobación"]
    C --> D["SITA: despliegue, pruebas y aprobación"]
    D --> E["Producción: código desplegado"]
    E --> F["Clientes o planes activados"]
    F --> G["Validación posterior y cierre"]
```

Un release no debería considerarse completado únicamente al llegar a producción. Debe existir una decisión documentada sobre activación y una validación posterior.

Rutina de control recomendada

Semanalmente con Ximena

Reunión de 15 minutos para revisar:

- Promociones realizadas.

- Pruebas y aprobaciones.
- Bloqueos técnicos.
- Releases planeados.
- Alcance de las release notes.
- Cambios de versión.
- Dependencias entre componentes.

Semanal o quincenalmente con Ameet

Revisión breve de:

- Despliegues pendientes en producción.
- Activación por planes o empresas.
- Fechas esperadas.
- Releases sin seguimiento.
- Problemas posteriores al lanzamiento.

Antes de cada promoción

Confirmar:

- Release ID.
- Componentes incluidos.
- Tickets incluidos.
- Release notes terminadas.
- Ambiente de destino.
- Responsable de la promoción.
- Responsable de QA.
- Fecha prevista.
- Dependencias resueltas.

Después de cada promoción

Registrar:

- Fecha real.
- Ambiente alcanzado.
- Resultado del despliegue.
- Inicio de pruebas.
- Evidencia.

- Próxima acción.

Después de producción

Confirmar:

- Versión desplegada.
- Fecha de despliegue.
- Activación.
- Planes o empresas habilitadas.
- Fecha del seguimiento.
- Feedback o incidentes.
- Criterio de cierre.

Responsabilidades sugeridas

Rol	Responsabilidad propuesta
Jhon	Mantener el tracker, coordinar seguimientos, registrar evidencias y escalar bloqueos
Ximena	Confirmar alcance técnico, dependencias y composición de releases
Desarrollo	Confirmar readiness, merge, tickets y contenido de release notes
QA	Confirmar pruebas, resultados y aprobación por ambiente
Ameet o responsable autorizado	Confirmar producción, fecha de activación y audiencia habilitada

Estas responsabilidades deberían validarse con el equipo; no se debe asumir que Jhon es responsable de desplegar o aprobar técnicamente.

Mi recomendación final

Implementaría el método en dos etapas:

1. Ahora: crear el Excel estructurado, cargar la línea base del 7 de agosto y resolver todas las confirmaciones pendientes.
2. Después de dos o tres sprints: revisar si el proceso puede apoyarse en Jira Fix Versions, Releases o un dashboard para reducir la actualización manual.

La regla principal sería:

> Ningún estado se actualiza sin fuente, fecha y evidencia; ninguna fila abierta permanece sin próxima acción, responsable y fecha de seguimiento.

El siguiente paso concreto sería crear el archivo Excel con las cuatro pestañas, listas desplegables, formato condicional y la información conocida ya precargada.

2026-08-14 17:09:41 **You**

Remittances formará parte de otro proyecto. No lo debemos incluir. Con la información que dispones. Me ayudas a crear el archivo Excel?

ChatGPT gpt-5.6-sol-wm

Sí. Crearé el Excel con Remittances completamente fuera del alcance. Usaré la matriz y las dos reuniones para precargar el estado conocido, estructurándolo como un control operativo con vista general, registro de releases, seguimientos e historial. También lo guardaré como un archivo reutilizable para que puedas descargarlo y mantenerlo.

La estructura ya está definida. El archivo mantendrá la matriz como vista ejecutiva, pero añadirá trazabilidad: cada elemento tendrá próximo paso, responsable del seguimiento, fecha, evidencia y separación entre producción técnica y activación para clientes. Remittances no aparecerá en ninguna pestaña.

La primera validación visual confirmó que la estructura funciona. Encontré una precisión importante y la estoy corrigiendo: Paola confirmó que ePay Improvements estaba en TATU, pero no confirmó aprobación de QA; por eso el estado correcto será “Deployed”, no “Approved”. También estoy ajustando el contraste de los indicadores del Overview.

El archivo está completo y validado. Incluye cinco pestañas: Overview, Release Register, Follow-ups, History y Lists; contiene la línea base del 7 de agosto y excluye completamente Remittances. Ahora estoy guardando el archivo final para entregártelo.

Listo, Jhon. Creé el archivo con:

- Línea base del 7 de agosto de 2026.
- Nueve unidades de release de Reports.
- Overview automático con indicadores.
- Registro detallado por ambiente.
- Nueve seguimientos iniciales.
- Historial de confirmaciones.
- Listas desplegables y estados estandarizados.

- Separación entre producción y activación para clientes.
- Remittances completamente excluido.

Las fechas objetivo y los Jira tickets desconocidos quedaron vacíos para evitar inventar información.

2026-08-14 17:42:27 **You**

Consideras apropiado cambiarlo para utilizar Loop Pages? Primero solo respondeme y analicemos este escenario. Te informo que algo similar debo hacer para otros proyectos. Debo controlar también AdminAPI, Dashboard, Invoicing, Fintrac

ChatGPT gpt-5.6-sol-wm

Voy a evaluar Loop como solución transversal para Reports, AdminAPI, Dashboard, Invoicing y Fintrac, comparándolo con el Excel que acabamos de crear. Por ahora solo analizaré el modelo y no modificaré el archivo. También verificaré las capacidades actuales de Loop para no basar la recomendación en funciones desactualizadas.

Sí, Loop Pages es apropiado para este escenario, especialmente porque debes controlar cinco proyectos. Pero no recomiendo reemplazar completamente el Excel por Loop.

La mejor solución sería híbrida:

- Loop como centro de coordinación, reuniones, decisiones y acciones.
- Excel como registro estructurado y auditable de releases.
- Jira como evidencia del trabajo técnico.

Microsoft define Loop precisamente como una combinación de workspaces, pages y componentes colaborativos para reunir información, tareas y personas. Además, sus componentes pueden compartirse y mantenerse sincronizados en Teams y Outlook. Microsoft: Get started with Loop

Por qué no usaría únicamente Loop

Loop sería excelente para:

- Tener una página por proyecto.
- Presentar el estado ejecutivo.
- Registrar decisiones de reuniones.
- Mantener release notes.
- Asignar acciones y fechas.

- Mencionar a Ximena, Ameet u otros responsables.
- Compartir el estado dentro de Teams.
- Dar seguimiento colaborativo.

Sin embargo, es menos adecuado para:

- Matrices extensas con muchos campos.
- Fórmulas e indicadores automáticos.
- Listas desplegables controladas.
- Formato condicional avanzado.
- Mantener un historial estructurado de cambios.
- Filtrar y comparar decenas de releases entre proyectos.
- Consolidar métricas de Reports, AdminAPI, Dashboard, Invoicing y Fintrac.

Por eso, trasladar íntegramente el archivo que creamos a una tabla de Loop implicaría perder parte de su capacidad de control.

Opciones

Opción	Evaluación
Loop solamente	Fácil de consultar y colaborar, pero débil como registro detallado y audit able
Excel solamente	Fuerte para datos y seguimiento, pero menos útil para reuniones, decisi ones y colaboración
Loop + un Excel por p royecto	Funcional, pero terminarías manteniendo cinco archivos y sería difícil o btener una visión global
Loop + Excel maestro + Jira	Recomendado: una vista global, páginas por proyecto y evidencia técni ca vinculada

Estructura recomendada

Crearía un único Loop Workspace, por ejemplo:

Product Delivery & Release Control

Dentro tendría estas páginas:

1. Portfolio Overview
2. Reports
3. AdminAPI

4. Dashboard
5. Invoicing
6. Fintrac
7. Decisions & Escalations

La página Portfolio Overview mostraría únicamente:

- Estado general de cada proyecto.
- Release activo.
- Ambiente alcanzado.
- Principal bloqueo.
- Próxima acción.
- Fecha del próximo seguimiento.
- Indicador de salud.

Ejemplo:

Proyecto	Release actual	Ambiente	Salud	Próxima acción
Reports	Summary Report	TATU Testing	Atención	Completar validación
AdminAPI	Por definir	Por confirmar	Sin datos	Revisar con Tech Lead
Dashboard	Por definir	Por confirmar	Sin datos	Construir línea base
Invoicing	Por definir	Producción	Seguimiento	Validar resultado
Fintrac	Por definir	Por confirmar	Sin datos	Construir línea base

Contenido de cada página de proyecto

Cada página utilizaría la misma plantilla:

Project Summary

- Objetivo.
- Tech Lead.
- Stakeholders.
- Última revisión.
- Estado general.
- Release activo.

Current Release Status

Una tabla pequeña, equivalente al Overview del Excel:

- Release Unit.
- MOJA.
- TATU.
- SITA.
- Production.
- Client Activation.
- Health.
- Next Action.

Active Release Package

- Release ID.
- Alcance.
- Tickets incluidos.
- Ambiente objetivo.
- Release notes.
- Dependencias.
- Fecha esperada.

Decisions

- Fecha.
- Decisión.
- Participantes.
- Impacto.
- Evidencia.

Follow-up Tasks

Aquí Loop sí aporta mucho valor. Sus listas de tareas permiten asignar responsables y fechas; además, se sincronizan con Microsoft Planner. En Loop solo se manejan directamente los campos básicos —título, responsable y fecha— y las funciones más completas se administran en Planner. Microsoft: Loop tasks and Planner

Meeting Notes

- Fecha de reunión.
- Participantes.

- Confirmaciones.
- Cambios de estado.
- Nuevos riesgos.
- Acciones.

Qué haría con el Excel actual

No lo descartaría. Lo convertiría en un registro maestro para todos los proyectos.

El principal cambio sería agregar una columna Project:

- Reports.
- AdminAPI.
- Dashboard.
- Invoicing.
- Fintrac.

El workbook tendría:

- Portfolio Overview: resumen de los cinco proyectos.
- Release Register: todas las unidades de release.
- Follow-ups: opcional, si las acciones todavía no se trasladan a Loop.
- History: historial de cambios.
- Lists: estados y reglas estandarizadas.

Después podrían agregarse vistas individuales por proyecto, pero todas utilizarían el mismo registro central.

Distribución de responsabilidades entre herramientas

Información	Herramienta principal
Estado por ambiente	Excel maestro
Historial de promociones	Excel maestro
Fechas y confirmaciones	Excel maestro
Tickets técnicos	Jira
Release notes	Loop o Jira, con enlace desde Excel
Resumen del proyecto	Loop Page
Notas de reuniones	Loop Page

Información	Herramienta principal
Decisiones	Loop Page
Acciones y recordatorios	Loop Task List/Planner
Conversación del equipo	Teams con componentes Loop

La regla importante sería evitar duplicación:

- El estado detallado se mantiene una sola vez en Excel.
- Loop presenta el resumen y el contexto.
- Jira conserva el detalle técnico.
- Las tareas se administran en Loop/Planner.
- Cada herramienta enlaza a las demás.

Beneficio para los cinco proyectos

Esta estructura permitiría que Jhon tenga una vista ejecutiva consolidada sin mezclar innecesariamente los detalles:

- Reports puede tener múltiples reportes y ambientes.
- AdminAPI puede tener APIs, migraciones y sincronizaciones.
- Dashboard puede tener módulos, feature toggles y widgets.
- Invoicing puede incluir problemas de producción y validaciones mensuales.
- Fintrac puede incluir funcionalidades, cumplimiento, pruebas y releases.

Todos utilizarían el mismo lenguaje de control:

- ¿Qué se está entregando?
- ¿Dónde está?
- ¿Quién lo confirmó?
- ¿Qué falta?
- ¿Quién debe actuar?
- ¿Cuándo se revisará nuevamente?
- ¿Está desplegado o realmente disponible?
- ¿Puede cerrarse?

Recomendación final

Usaría Loop, pero como capa de coordinación y portfolio, no como reemplazo del registro de releases.

La arquitectura recomendada sería:

- Un Loop Workspace para los cinco proyectos.
- Una página estandarizada por proyecto.
- Un Excel maestro para todo el portafolio.
- Jira como fuente de tickets y evidencia técnica.
- Loop/Planner para acciones, responsables y fechas.

Antes de migrar, conviene confirmar que tu organización permite crear Loop workspaces y páginas, porque la disponibilidad depende de las políticas y licencias configuradas por TI. Microsoft: Loop access and licensing

Mi recomendación sería probar este modelo primero con Reports durante uno o dos sprints. Si funciona, se replica la misma plantilla en AdminAPI, Dashboard, Invoicing y Fintrac.

2026-08-14 17:46:44 **You**

Te parecería mejor desarrollar un sistema? por el momento solo analicemos esta opción. Podríamos elaborar el PRD del nuevo sistema? definir los requerimientos?

ChatGPT gpt-5.6-sol-wm

Voy a analizar el sistema como una alternativa independiente, sin asumir todavía que construirlo sea la decisión correcta. Usaré lo aprendido con Reports y el Excel como evidencia del proceso real: estados, releases agrupados, aprobaciones, activaciones y seguimientos. No modificaré ningún archivo por ahora.

Sí, desarrollar un sistema podría ser una mejor solución a largo plazo, especialmente porque el control no se limita a Reports: también necesitas administrar AdminAPI, Dashboard, Invoicing y Fintrac.

Sin embargo, todavía no recomendaría comenzar a programarlo. Primero elaboraría un PRD neutral respecto a la tecnología, definiría los requerimientos y después evaluaría si conviene:

- Desarrollar una aplicación propia.
- Construir una solución low-code.
- Extender Jira.
- Mantener un modelo híbrido con Loop y Excel.

¿Se justifica un sistema?

Tu necesidad ya no es simplemente “mantener una matriz”. En realidad, necesitas controlar un portafolio con:

- Cinco proyectos.
- Múltiples productos y módulos.
- Releases que agrupan varios tickets.
- Promociones entre ambientes.
- Pruebas y aprobaciones.
- Dependencias.
- Producción.
- Activación por planes o empresas.
- Seguimientos y recordatorios.
- Historial y evidencia.
- Participación de Tech Leads, QA, desarrolladores y Ameet.

Esto sí constituye un problema suficientemente definido como para evaluar un sistema.

El caso de Reports ya identificó situaciones que una herramienta debería resolver:

- Código validado en MOJA que nunca fue promovido.
- Producto presente en producción pero no activado.
- Versión base y mejoras en ambientes diferentes.
- Releases potencialmente agrupados.
- Decisiones dispersas en mensajes.
- Estados conocidos por distintas personas.
- Falta de seguimiento después de solicitar una acción.
- Dificultad para saber cuándo una iniciativa puede cerrarse.

Principal riesgo

El mayor riesgo no es técnico. Es construir una herramienta adicional que nadie mantenga y que termine compitiendo con Jira, Teams, Excel y los sistemas de deployment.

El sistema debe evitar convertirse en otro lugar donde copiar manualmente toda la información.

Por eso debemos definir desde el PRD:

- Qué información será propia del sistema.
- Qué información seguirá viviendo en Jira.
- Qué información podría importarse automáticamente.
- Quién actualizará cada estado.
- Qué evento desencadena una actualización.

- Cuál será la fuente oficial de cada dato.

Opciones de implementación

Alternativa	Ventaja	Riesgo
Excel + Loop	Implementación inmediata y bajo costo	Actualización manual y menor trazabilidad
Jira configurado para releases	Reutiliza tickets y usuarios existentes	Puede no representar bien activación, ambientes y seguimiento ejecutivo
Solución low-code	Desarrollo más rápido y formularios estructurados	Dependencia de licencias y límites de la plataforma
Sistema web propio	Flexibilidad, automatización e integraciones	Mayor costo, mantenimiento y soporte
Modelo híbrido	Sistema para releases y Jira para desarrollo	Requiere una frontera clara entre herramientas

Mi hipótesis inicial sería evaluar primero una solución low-code o una extensión sobre herramientas existentes. Una aplicación completamente personalizada debería justificarse después del PRD.

El Excel actual sigue siendo útil

El archivo de Reports no sería trabajo descartado. Funciona como:

- Prototipo del sistema.
- Primera definición del modelo de datos.
- Evidencia de los flujos reales.
- Línea base para los requerimientos.
- Conjunto inicial de casos de prueba.
- Fuente para identificar excepciones.

Cada pestaña ya representa un posible módulo:

Excel	Posible módulo del sistema
Overview	Portfolio Dashboard
Release Register	Release Management
Follow-ups	Actions and Notifications
History	Audit Log
Lists	Configuration and Business Rules

Posible alcance del sistema

Un nombre provisional podría ser:

Project and Release Control System

Su objetivo inicial:

> Centralize the status and follow-up of projects, modules and releases from development through production activation and closure, providing ownership, evidence and an auditable history.

Modelo conceptual

La estructura principal podría ser:

Project → Product/Module → Release → Release Item → Environment Status → Client Activation → Post-release Closure

Ejemplo:

- Project: Reports
- Module: ePay Report
- Release: August 2026 Release
- Release Item: AdminAPI/Rejections/Dynamics Improvements
- Current Environment: TATU
- Environment Status: Deployed
- Next Environment: SITA
- Next Action: Confirm QA result and release notes
- Follow-up Owner: Jhon
- Technical Confirmation: Paola
- Activation: Not applicable yet

Módulos potenciales del MVP

1. Portfolio Dashboard

- Todos los proyectos.
- Estado general.
- Release activo.
- Ambiente actual.
- Bloqueos.
- Acciones vencidas.

- Próximas promociones.
- Elementos pendientes de producción o activación.

2. Project and Module Management

- Proyectos.
- Productos.
- Módulos.
- Tech Lead.
- Stakeholders.
- Prioridad.
- Estado general.
- Relación entre proyectos.

3. Release Management

- Release ID.
- Nombre.
- Alcance.
- Ambiente origen y destino.
- Fecha esperada.
- Release notes.
- Tickets incluidos.
- Responsable técnico.
- Responsable de QA.
- Estado.
- Dependencias.

4. Release Items

Permitirá controlar componentes que pertenecen al mismo release, pero pueden tener estados diferentes.

Por ejemplo:

- Summary Report.
- Cross-cutting Improvements.
- ePay Improvements.

Pueden compartir un Release ID sin perder su estado individual.

5. Environment Tracking

Para cada ambiente:

- Presencia confirmada.
- Fecha de despliegue.
- Estado de pruebas.
- Resultado.
- Aprobación.
- Confirmado por.
- Evidencia.
- Bloqueo.
- Próximo ambiente.

Los ambientes deberían ser configurables porque no todos los proyectos necesariamente utilizan exactamente MOJA, TATU, SITA y Production.

6. Production and Client Activation

Debe separar:

- Producción técnica.
- Versión desplegada.
- Fecha de despliegue.
- Activación.
- Planes habilitados.
- Empresas habilitadas.
- Piloto.
- Disponibilidad general.
- Fecha de verificación.
- Feedback posterior.

7. Actions and Follow-ups

- Acción.
- Responsable del seguimiento.
- Persona o equipo contactado.
- Fecha objetivo.
- Estado.

- Recordatorio.
- Resultado.
- Evidencia.
- Escalación.

8. Audit History

- Cambio.
- Estado anterior.
- Estado nuevo.
- Usuario.
- Fecha.
- Evidencia.
- Comentario.

El historial no debería poder alterarse silenciosamente.

9. Reports and Filters

- Por proyecto.
- Por Tech Lead.
- Por ambiente.
- Por responsable.
- Por estado.
- Releases bloqueados.
- Acciones vencidas.
- Producción sin activación.
- Registros sin verificación reciente.
- Iniciativas sin próxima acción.

10. Jira Integration

Para el MVP podría comenzar únicamente con enlaces a Jira.

En fases posteriores:

- Consultar títulos y estados.
- Importar tickets por Release ID o Fix Version.
- Detectar tickets abiertos dentro de un release.

- Evitar copiar manualmente información técnica.
- Mostrar discrepancias entre Jira y el release registrado.

Roles iniciales

Rol	Capacidades
Portfolio/Business Analyst	Crear proyectos, coordinar releases, administrar acciones y generar reportes
Tech Lead	Confirmar alcance, dependencias y readiness técnico
Developer	Actualizar desarrollo, merge y release notes
QA	Registrar pruebas, resultados y aprobaciones
Production Approver	Confirmar despliegue y activación
Stakeholder	Consultar estado y agregar comentarios
Administrator	Configurar ambientes, estados, usuarios y permisos

Estos roles no deben implicar que todos necesariamente actualizarán el sistema; eso debe validarse durante el discovery.

Requerimientos no funcionales

El PRD también debería contemplar:

- Acceso basado en roles.
- Autenticación corporativa.
- Historial auditable.
- Protección de información de producción.
- Backups.
- Exportación a Excel.
- Integración con Jira.
- Notificaciones.
- Diseño responsive.
- Accesibilidad.
- Rendimiento.
- Retención de información.
- Configuración por proyecto.

Qué dejaría fuera del MVP

Para evitar sobredimensionarlo, inicialmente excluiría:

- Ejecución automática de deployments.
- Acceso directo a datos de clientes.
- Modificación automática de tickets Jira.
- Gestión completa de sprints.
- Reemplazo de Jira.
- Administración de código fuente.
- Analítica predictiva.
- Integración con todos los sistemas al mismo tiempo.

El sistema debe controlar y orquestar releases, no convertirse en una nueva plataforma de desarrollo.

Estructura propuesta del PRD

Podemos elaborar un PRD completo con:

1. Executive Summary.
2. Background and Problem Statement.
3. Product Vision.
4. Goals.
5. Non-goals.
6. Users and Roles.
7. Current Process.
8. Pain Points.
9. Proposed Workflow.
10. Functional Requirements.
11. Business Rules.
12. Data Model.
13. Notifications and Escalations.
14. Jira and Microsoft 365 Integrations.
15. Security and Permissions.
16. Reporting Requirements.
17. Non-functional Requirements.
18. MVP Scope.

19. Future Phases.
20. Success Metrics.
21. Risks and Dependencies.
22. Open Questions.
23. Acceptance Criteria.

Recomendación

Sí recomiendo elaborar el PRD. Todavía no recomiendo decidir que necesariamente será una aplicación web personalizada.

El orden correcto sería:

1. Utilizar Reports como caso inicial.
2. Recopilar las particularidades de AdminAPI, Dashboard, Invoicing y Fintrac.
3. Identificar qué elementos son comunes y cuáles son específicos.
4. Definir el proceso objetivo.
5. Elaborar el PRD.
6. Diseñar un prototipo.
7. Comparar Jira, low-code y desarrollo propio.
8. Definir el MVP.
9. Implementar un piloto con Reports.
10. Evaluar el resultado antes de incorporar los otros proyectos.

Con la información actual ya tenemos suficiente para comenzar el PRD, pero primero debería construirse como un producto para los cinco proyectos, usando Reports como primer caso de uso, no como una herramienta exclusiva de Reports.

2026-08-14 17:53:21 **You**

Se podría utilizar herramientas automáticas de Microsoft en lugar de desarrollar un sistema?

ChatGPT gpt-5.6-sol-wm

Sí, esta puede ser la alternativa más conveniente antes de considerar desarrollo propio. Voy a evaluar una arquitectura basada en Microsoft Lists/SharePoint, Power Automate, Power Apps, Power BI,

Teams y Loop, verificando qué parte del proceso podría automatizar cada herramienta y qué limitaciones o licencias habría que confirmar.
Sí. De hecho, antes de desarrollar un sistema propio, recomendaría evaluar una solución basada en Microsoft 365 y Power Platform. Para tu escenario, puede ofrecer aproximadamente la misma experiencia operativa con menos desarrollo y mantenimiento.

La arquitectura más razonable sería:

- Microsoft Lists/SharePoint: base de datos central.
- Power Automate: recordatorios, aprobaciones, alertas e historial.
- Power Apps: interfaz opcional para consultar y actualizar releases.
- Power BI: dashboard ejecutivo.
- Teams: punto de acceso para el equipo.
- Loop: reuniones, decisiones y contexto.
- Jira: fuente del trabajo técnico.

Arquitectura recomendada

1. Microsoft Lists como fuente central

En lugar de utilizar Excel como base principal, crearía listas relacionadas:

Projects

- Project ID.
- Project Name.
- Business Analyst.
- Tech Lead.
- Priority.
- Overall Health.
- Last Review.
- Next Review.

Registros iniciales:

- Reports.
- AdminAPI.
- Dashboard.
- Invoicing.
- Fintrac.

Release Units

Equivale al Release Register del Excel:

- Release ID.
- Project.
- Product/Module.
- Release Unit/Scope.
- Priority.
- Jira Tickets.
- Development Owner.
- QA Owner.
- MOJA Status.
- TATU Status.
- SITA Status.
- Production Deployment.
- Client Activation.
- Dependencies.
- Next Action.
- Follow-up Owner.
- Follow-up Date.
- Last Verified.
- Confirmed By.
- Health.
- Evidence.

Release Packages

Para agrupar elementos que se promoverán juntos:

- Package ID.
- Project.
- Target Environment.
- Planned Date.
- Release Notes.
- Included Release Units.

- Readiness.
- Approval Status.
- Blockers.

Follow-ups

- Action ID.
- Release ID.
- Action.
- Owner.
- Contact.
- Due Date.
- Status.
- Result.
- Evidence.

Release History

- Event Date.
- Release ID.
- Environment.
- Event Type.
- Previous Status.
- New Status.
- Confirmed By.
- Evidence.

Microsoft Lists permite crear reglas sencillas para enviar notificaciones cuando se crea o modifica un elemento. Para automatizaciones más avanzadas, Microsoft recomienda Power Automate, Power Apps y Power BI. Microsoft: Automate a list

2. Power Automate para el control automático

Aquí estaría el mayor beneficio frente al Excel.

Recordatorios de seguimiento

Un flujo diario podría buscar:

- Acciones vencidas.
- Seguimientos que vencen próximamente.

- Releases sin actualización reciente.
- Producción sin activación confirmada.
- Filas sin próxima acción.
- Elementos bloqueados.

Power Automate admite recordatorios basados en columnas de fecha de Microsoft Lists/SharePoint. Microsoft: Create a reminder flow

Notificaciones por cambios de estado

Ejemplos:

- Cuando algo llegue a TATU, notificar a QA.
- Cuando QA cambie el estado a Approved, notificar a Ximena.
- Cuando una promoción quede bloqueada, notificar a Jhon.
- Cuando algo llegue a producción, crear un seguimiento de activación.
- Cuando se confirme Client Activation, programar validación posterior.
- Cuando una acción sea completada, actualizar la salud del release.

Aprobaciones formales

Podrían crearse flujos para solicitar:

- Aprobación de QA.
- Aprobación para promover a SITA.
- Confirmación de producción.
- Confirmación de activación.
- Aprobación de release notes.

Power Automate puede enviar solicitudes de aprobación por correo o mediante su centro de aprobaciones y actualizar automáticamente el registro con la respuesta y comentarios. Microsoft: Create an approval workflow

Resumen semanal automático

Cada semana podría generarse y enviarse un resumen con:

- Releases activos por proyecto.
- Acciones vencidas.
- Elementos bloqueados.
- Promociones realizadas.
- Promociones esperadas.

- Producción pendiente de activación.
- Registros sin verificar.

3. Power Apps como interfaz

Microsoft Lists podría utilizarse directamente durante el piloto. Si las listas resultan difíciles de navegar, se podría crear una Power App.

Power Apps permite generar una aplicación web y móvil conectada directamente a Microsoft Lists/SharePoint; los cambios realizados desde la aplicación actualizan la lista central. Microsoft: Create a canvas app from a list

La aplicación podría tener estas pantallas:

- Portfolio Dashboard.
- Projects.
- Active Releases.
- Release Details.
- Environment Progress.
- Follow-ups.
- Approvals.
- History.
- My Actions.

También podría limitar los campos según el rol:

- QA actualiza pruebas y aprobaciones.
- Desarrollo actualiza readiness, merge y release notes.
- Tech Lead confirma dependencias.
- Jhon mantiene prioridades, acciones y seguimiento.
- Ameet confirma producción y activación.

Esto se comportaría como un sistema desarrollado, pero sin construir Backend y Frontend desde cero.

4. Power BI para reportes

Power BI podría conectarse a las listas de SharePoint para crear:

- Releases por ambiente.
- Releases bloqueados por proyecto.
- Tiempo promedio entre ambientes.
- Producción sin activación.

- Acciones vencidas por responsable.
- Releases sin actualización reciente.
- Evolución mensual de promociones.
- Salud general de cada proyecto.

Microsoft proporciona un conector para cargar información de SharePoint Online Lists en Power BI. Microsoft: Create a report on a SharePoint Online list

No lo incorporaría inicialmente. Primero debemos tener datos consistentes.

5. Teams como punto de entrada

El equipo podría acceder desde Teams a:

- La Power App.
- Las listas.
- El dashboard.
- La página Loop.
- Las acciones.
- Las release notes.

Esto evitaría pedirle al equipo que ingrese a una aplicación completamente nueva.

6. Loop como contexto, no como base de datos

Mantendría Loop para:

- Notas de reuniones.
- Decisiones.
- Riesgos.
- Resumen ejecutivo.
- Información explicativa.
- Enlaces a releases.
- Agenda de seguimiento.

Las tablas operativas quedarían en Microsoft Lists.

Así evitamos duplicar manualmente los mismos estados entre Loop y la solución.

7. Integración con Jira

Power Platform tiene un conector oficial para Jira que puede consultar issues mediante JQL y ejecutar diferentes acciones. Sin embargo, es un conector Premium y tiene limitaciones técnicas y de autenticación que deben evaluarse. Microsoft: Jira connector

Por eso lo implementaría en dos fases:

MVP

- Guardar Jira Key.
- Guardar enlace al ticket.
- Consultar Jira manualmente.
- No actualizar tickets desde Microsoft.

Fase posterior

- Importar título, estado, prioridad y assignee.
- Detectar tickets abiertos dentro de un release.
- Mostrar discrepancias.
- Actualizar automáticamente la última sincronización.

No permitiría inicialmente que Power Automate modificara tickets, porque aumentaría el riesgo y la complejidad.

Tres niveles posibles

Nivel 1 — Microsoft Lists + Teams

Incluye:

- Listas estructuradas.
- Vistas por proyecto.
- Reglas básicas.
- Historial.
- Filtros.
- Notificaciones sencillas.

Es el piloto más rápido y de menor complejidad.

Nivel 2 — Lists + Power Automate

Añade:

- Recordatorios.
- Alertas de vencimiento.
- Aprobaciones.
- Historial automático.
- Resumen semanal.

- Seguimientos de producción.
- Escalaciones.

Este sería mi MVP recomendado.

Nivel 3 — Power Apps + Power BI + integraciones

Añade:

- Interfaz personalizada.
- Dashboard ejecutivo.
- Experiencia basada en roles.
- Integración con Jira.
- Analítica consolidada.
- Automatización completa.

Se implementaría después de validar el proceso.

Ventajas frente a desarrollar un sistema

Aspecto	Power Platform	Sistema propio
Implementación	Más rápida	Más extensa
Autenticación	Microsoft 365	Debe desarrollarse
Permisos	SharePoint/Microsoft	Deben diseñarse
Automatización	Power Automate	Código personalizado
Interfaz	Power Apps	Frontend propio
Reportes	Power BI	Desarrollo propio
Mantenimiento	Menor	Mayor
Flexibilidad	Media/alta	Muy alta
Jira	Conector Premium	Integración propia
Dependencia tecnológica	Microsoft	Equipo de desarrollo

Riesgos y condiciones

Antes de elegir esta alternativa deben confirmarse con TI:

- Acceso a SharePoint y Microsoft Lists.
- Licencias de Power Apps.

- Licencias de Power Automate.
- Permiso para crear y publicar flows.
- Disponibilidad de Power BI.
- Disponibilidad del conector Premium de Jira.
- Políticas de protección de datos.
- Cuenta propietaria de las automatizaciones.
- Responsabilidad de mantenimiento.
- Permisos de los diferentes equipos.

Es especialmente importante que los flujos no dependan únicamente de la cuenta personal de Jhon. Deben tener una estrategia de propiedad, respaldo y soporte.

Recomendación

Esta sería ahora mi primera opción:

1. Elaborar el PRD funcional, sin asumir desarrollo propio.
2. Definir Microsoft Power Platform como solución candidata.
3. Construir un piloto solamente para Reports.
4. Usar Microsoft Lists como fuente central.
5. Implementar tres o cuatro flujos esenciales:
    - Acciones vencidas.
    - Registro sin actualización.
    - Producción pendiente de activación.
    - Resumen semanal.
6. Probarlo durante dos sprints.
7. Ajustar el modelo.
8. Incorporar AdminAPI, Dashboard, Invoicing y Fintrac.
9. Agregar Power Apps solamente si las listas no ofrecen una experiencia suficiente.
10. Evaluar Power BI e integración con Jira después de estabilizar los datos.

Sí podemos elaborar un PRD para esta solución. El PRD debería describir primero el problema y los requerimientos, y luego presentar Microsoft Lists + Power Automate como la implementación recomendada para el MVP.

2026-08-14 18:03:49 **You**

No tengo experiencia en el uso de las herramientas de Microsoft, cómo lo puedo solventar?

ChatGPT gpt-5.6-sol-wm

No necesitas convertirte en especialista de toda la Power Platform. Voy a proponerte una ruta de aprendizaje mínima y una forma de dividir responsabilidades para que puedas liderar la solución como Business Analyst sin asumir solo la construcción técnica. También revisaré los recursos oficiales actuales para que el aprendizaje sea práctico y progresivo.

Lo puedes solventar sin convertirte en experto técnico. Por tu experiencia como Business Analyst e ingeniero, tu rol debería ser diseñar el proceso, los datos y las reglas; no necesariamente construir y administrar solo toda la solución.

Mi recomendación es una estrategia combinada:

- Aprender únicamente lo necesario.
- Empezar con Microsoft Lists, no con toda la Power Platform.
- Trabajar con alguien que conozca Microsoft 365 para la configuración inicial.
- Implementar Reports como piloto.
- Incorporar automatizaciones gradualmente.

No intentes aprender todo al mismo tiempo

Para el MVP no necesitas inicialmente:

- Power Apps.
- Power BI.
- Dataverse.
- Integración con Jira.
- Automatizaciones complejas.
- Aplicaciones móviles.
- Copilot Studio.

El primer piloto podría funcionar solamente con:

1. Microsoft Lists.
2. Reglas básicas.
3. Un flujo sencillo en Power Automate.
4. Acceso desde Teams.

Después se añaden las demás herramientas únicamente si aportan valor.

Qué deberías aprender tú

Microsoft Lists

Necesitas conocer:

- Crear una lista.
- Crear columnas.
- Tipos de datos.
- Listas desplegables.
- Columnas de persona.
- Columnas de fecha.
- Vistas y filtros.
- Formato condicional.
- Permisos.
- Historial de versiones.
- Importación desde Excel.

Lists será conceptualmente familiar porque se parece al Excel que acabamos de crear.

Power Automate

Inicialmente solo necesitas comprender:

- Trigger: qué inicia un flujo.
- Action: qué debe hacer.
- Condition: bajo qué condición actúa.
- Scheduled flow: flujo ejecutado periódicamente.
- Notification: mensaje o correo.
- Update item: actualización de un registro.
- Test history: resultado de las ejecuciones.

Microsoft tiene una ruta introductoria sin prerrequisitos que enseña a crear, modificar, probar y monitorear un flujo. Microsoft Learn: Power Automate fundamentals

Una ruta más completa, de aproximadamente cinco horas, cubre workflows y administración de flujos. Microsoft Learn: Automate a business process

Power Apps

No necesitas aprenderlo hasta comprobar que Microsoft Lists no ofrece una experiencia suficientemente amigable.

Cuando llegue el momento, existe una ruta para principiantes orientada a Business Users, Functional Consultants y App Makers. Microsoft Learn: Create a Canvas App

Power BI

Déjalo para el final. Un dashboard no será útil hasta que los estados y fechas se actualicen consistentemente.

Modelo de responsabilidades recomendado

Participante	Responsabilidad
Jhon	PRD, proceso, campos, reglas, prioridades, criterios de aceptación y UAT
Especialista Power Platform	Arquitectura, listas, flows, permisos, conexiones y publicación
IT/Microsoft 365 Admin	Licencias, ambientes, seguridad, políticas y cuentas
Tech Leads	Validar el flujo técnico y los estados
QA	Validar pruebas, aprobaciones y evidencias
Stakeholders	Validar dashboards, reportes y alertas

Así no dependes completamente de otra persona, pero tampoco asumes solo una tecnología nueva.

Opciones para obtener ayuda

1. Power Platform Champion interno

Pregunta si PaymentEvolution tiene:

- Microsoft 365 Administrator.
- SharePoint Administrator.
- Power Platform Administrator.
- Power Apps developer.
- Automation specialist.
- Center of Excellence.
- Empleado que ya mantenga flows.

Esta sería la mejor opción porque esa persona conocerá las licencias, políticas y restricciones internas.

2. Trabajo en pareja

Una persona con experiencia puede construir la estructura inicial mientras tú:

- Defines el modelo.
- Explicas los casos de negocio.
- Verificas los estados.
- Escribes los criterios de aceptación.
- Ejecutas UAT.
- Documentas el proceso.

Durante la construcción puedes aprender cómo mantenerla.

3. Formación “In a Day”

Microsoft ofrece talleres como:

- Automation in a Day.
- App in a Day.

Están orientados a construir una solución práctica con acompañamiento. La ruta oficial de Power Automate menciona el taller Automation in a Day como alternativa de aprendizaje guiado. Microsoft Learn: Power Automate training

4. Proveedor o consultor externo

Solamente lo consideraría si:

- No existe conocimiento interno.
- La solución tendrá muchos usuarios.
- Se necesita integración con Jira.
- Existen reglas estrictas de seguridad.
- El piloto debe publicarse rápidamente.

Ruta práctica de aprendizaje e implementación

Etapa 1 — Preparación

Confirma con IT:

- ¿Tengo acceso a Microsoft Lists?
- ¿Puedo crear listas en un sitio de SharePoint o Team?
- ¿Tengo acceso a Power Automate?
- ¿Puedo crear y compartir flows?
- ¿Existe un ambiente de prueba?

- ¿Quién debe ser copropietario de la solución?
- ¿Qué licencias están disponibles?
- ¿Está permitido utilizar Power Apps?
- ¿Está disponible el conector de Jira?

Etapa 2 — Piloto manual con Reports

Crear únicamente:

- Projects.
- Release Register.
- Follow-ups.
- Release History.

Importar la información del Excel y utilizarla manualmente durante una o dos revisiones con Ximena.

Todavía no enviar notificaciones al equipo.

Etapa 3 — Primera automatización

Crear un solo flujo:

> Cuando una acción tenga fecha vencida y no esté completada, enviar un recordatorio únicamente a Jhon.

Esto permite aprender y probar sin impactar a otras personas.

Etapa 4 — Automatizaciones controladas

Añadir progresivamente:

1. Registro sin actualización reciente.
2. Release bloqueado.
3. Producción pendiente de activación.
4. Seguimiento posterior a producción.
5. Resumen semanal.

Cada flujo debe probarse individualmente antes de habilitar el siguiente.

Etapa 5 — Colaboración

Después del piloto:

- Compartir la lista con Ximena.
- Definir qué campos puede actualizar cada rol.
- Incorporar QA.

- Agregar notificaciones de Teams.
- Documentar quién mantiene la solución.

Etapa 6 — Expansión

Cuando Reports esté estable:

- Incorporar AdminAPI.
- Incorporar Dashboard.
- Incorporar Invoicing.
- Incorporar Fintrac.
- Evaluar Power Apps.
- Evaluar Power BI.
- Evaluar Jira.

Medidas de seguridad para el aprendizaje

Durante el piloto:

- No utilizar información sensible de clientes.
- No conectarse inicialmente a producción.
- No modificar tickets de Jira automáticamente.
- No enviar mensajes a grupos completos.
- Probar los flows únicamente con tu correo.
- Mantener un copropietario.
- Documentar cada automatización.
- Definir cómo desactivar cada flujo.
- Registrar errores y ejecuciones fallidas.

Cómo puedo ayudarte

Podemos trabajar juntos de manera progresiva:

1. Elaborar el PRD.
2. Diseñar el modelo de Microsoft Lists.
3. Definir cada columna y relación.
4. Crear un documento de arquitectura.
5. Diseñar los flows en lenguaje funcional.
6. Redactar historias o tickets para implementación.
7. Preparar casos de prueba.

8. Crear el plan de UAT.
9. Elaborar una guía paso a paso para que configures el piloto.
10. Analizar capturas de pantalla y errores mientras lo construyes.

La mejor solución no es que aprendas toda la plataforma antes de comenzar. Es que construyamos un piloto pequeño, aprendas sobre una necesidad real y obtengas apoyo técnico únicamente en las áreas donde sea necesario.

2026-08-14 18:08:54 **You**

Tengo experiencia en el desarrollo de aplicaciones con Codex utilizando MySql como Base de datos y Angular, consideras como una alternativa el desarrollo utilizando estas herramientas?

ChatGPT gpt-5.6-sol-wm

Sí, tu experiencia cambia materialmente la decisión: Angular y MySQL hacen que una solución propia sea mucho más viable para ti que para alguien que empezaría desde cero. Voy a analizar qué piezas adicionales necesitarías —especialmente Backend, autenticación, hosting, auditoría y soporte— y comparar esta ruta con Power Platform, sin proponer todavía que empecemos a construir.

Sí. Con tu experiencia en Angular, MySQL y desarrollo asistido por Codex, una aplicación propia es una alternativa realista y competitiva frente a Power Platform.

Mi recomendación técnica preliminar sería un modelo híbrido:

- Aplicación propia para el control de proyectos y releases.
- Angular para Frontend.
- NestJS/Node.js para Backend.
- MySQL para persistencia.
- Microsoft Entra ID para autenticación corporativa.
- Jira REST API para consultar tickets.
- Power Automate o Teams únicamente para notificaciones.

Esto aprovecha lo que ya conoces sin renunciar a las capacidades corporativas de Microsoft.

La pieza que falta: Backend

Angular y MySQL no deberían conectarse directamente. Necesitarías una API entre ambos:

```
text
```

```
Angular → REST API → MySQL
                   → Jira
                   → Microsoft/Teams
```

Como ya trabajarías con TypeScript en Angular, NestJS sería una opción coherente para el Backend:

- Mantiene el mismo lenguaje.
- Tiene arquitectura modular.
- Facilita controladores, servicios y validaciones.
- Admite autenticación y autorización.
- Permite pruebas unitarias e integración.
- Puede conectarse con MySQL mediante TypeORM, Sequelize u otras alternativas.

La documentación oficial de NestJS describe integración directa con MySQL mediante TypeORM o Sequelize y recomienda utilizar migraciones para evolucionar el esquema sin perder información.
NestJS: Database integration

Angular sigue siendo una buena base para aplicaciones empresariales estructuradas y escalables.
Angular: Official overview

Arquitectura propuesta

Frontend — Angular

Responsable de:

- Portfolio Dashboard.
- Gestión de proyectos.
- Registro de releases.
- Matriz por ambientes.
- Follow-ups.
- Historial.
- Administración de usuarios y roles.
- Filtros y reportes.
- Formularios de confirmación y aprobación.

Backend — NestJS

Responsable de:

- Reglas de negocio.
- Seguridad.

- Autorización.
- Validaciones.
- Persistencia.
- Historial auditable.
- Recordatorios.
- Integración con Jira.
- Notificaciones.
- Cálculo de salud.
- Control de transiciones.
- Exportación a Excel.
- API para Angular.

Base de datos — MySQL

Entidades iniciales:

- Projects.
- Modules.
- Releases.
- Release Items.
- Environments.
- Environment Statuses.
- Release Transitions.
- Release Packages.
- Follow-up Actions.
- Client Activations.
- Evidence.
- Audit Logs.
- Users.
- Roles.
- Notifications.

Modelo de datos conceptual

Project

Ejemplos:

- Reports.
- AdminAPI.
- Dashboard.
- Invoicing.
- Fintrac.

Module

Ejemplos de Reports:

- Payrun History.
- ePay Report.
- Summary Report.
- Cheques & Payslips.
- PayExplorer.
- Reports Homepage.

Release

Representa una entrega agrupada:

- Release ID.
- Nombre.
- Proyecto.
- Ambiente objetivo.
- Fecha planeada.
- Release notes.
- Estado.
- Responsable.

Release Item

Representa cada alcance que se mueve dentro del release:

- Summary Report.
- Cross-cutting Improvements.
- ePay Improvements.

Esto permite agruparlos sin perder sus estados individuales.

Release Transition

Cada cambio entre ambientes:

- Release Item.
- Ambiente origen.
- Ambiente destino.
- Estado.
- Fecha solicitada.
- Fecha desplegada.
- Fecha aprobada.
- Confirmado por.
- Evidencia.
- Bloqueo.

Client Activation

Separado de producción:

- Versión desplegada.
- Fecha de producción.
- Estado de activación.
- Planes.
- Empresas.
- Fecha de piloto.
- Fecha de disponibilidad general.
- Fecha de validación.
- Resultado.

Follow-up Action

- Acción.
- Responsable del seguimiento.
- Persona contactada.
- Fecha objetivo.
- Estado.
- Resultado.
- Evidencia.
- Escalación.

Audit Log

- Usuario.
- Entidad.
- Acción.
- Estado anterior.
- Estado nuevo.
- Fecha.
- Origen.
- Comentario.

El audit log debería generarse desde la aplicación; no dependería únicamente del historial técnico de MySQL.

Autenticación corporativa

No desarrollaría usuarios y contraseñas propios.

Si PaymentEvolution utiliza Microsoft 365, la aplicación Angular podría autenticarse mediante Microsoft Entra ID. Microsoft proporciona un flujo oficial para iniciar sesión desde una Angular SPA utilizando MSAL y autorización con PKCE. Microsoft: Angular SPA authentication

Después, el Backend validaría el token y aplicaría roles como:

- Administrator.
- Business Analyst.
- Tech Lead.
- Developer.
- QA.
- Production Approver.
- Viewer.

NestJS también proporciona mecanismos para proteger rutas mediante guards y tokens. NestJS: Authentication

Integración con Jira

La primera versión podría guardar únicamente:

- Jira Key.
- URL.
- Título.
- Estado sincronizado.

- Fecha de última sincronización.

Posteriormente, el Backend podría consultar Jira mediante JQL para:

- Obtener tickets de un proyecto.
- Consultar tickets de un release.
- Detectar tickets abiertos.
- Importar estado, prioridad y assignee.
- Mostrar discrepancias.

Atlassian dispone de una API REST para buscar issues mediante JQL y solo devuelve aquellos que la cuenta de integración tiene permiso para visualizar. Atlassian: Jira issue search API

Inicialmente la integración debería ser de solo lectura. No permitiría crear, modificar o cerrar tickets automáticamente.

Automatizaciones propias

El Backend podría ejecutar procesos programados:

- Detectar acciones vencidas.
- Detectar registros sin actualización.
- Detectar producción sin activación.
- Generar seguimientos posteriores.
- Calcular el Health.
- Enviar resumen semanal.
- Notificar bloqueos.
- Recordar aprobaciones.
- Cerrar releases cuando se cumplan los criterios.

Las notificaciones podrían enviarse mediante:

- Correo.
- Teams.
- Power Automate.
- Notificaciones internas de la aplicación.

Usar Power Automate solamente para notificaciones sería mucho más sencillo que construir toda la solución sobre Power Platform.

Comparación

Aspecto	Angular/NestJS/MySQL	Power Platform
Experiencia actual de Jhon	Alta	Baja
Flexibilidad	Muy alta	Media/alta
Modelo de datos	Totalmente controlable	Condicionado por Lists/Dataverse
Interfaz	Totalmente personalizada	Configurable con limitaciones
Automatizaciones	Requieren desarrollo	Más rápidas de configurar
Integración con Jira	API propia	Conector Premium
Auditoría	Debe implementarse	Algunas capacidades incluidas
Hosting	Debe resolverse	Gestionado por Microsoft
Seguridad	Debe diseñarse	Integrada con la plataforma
Mantenimiento	Responsabilidad del equipo	Menor, pero dependiente de licencias
Costos	Desarrollo e infraestructura	Licencias y administración
Dependencia de una persona	Riesgo elevado	También posible si los flows son personales

Ventajas específicas para ti

- Trabajarías con herramientas conocidas.
- Codex puede acelerar scaffolding, formularios, endpoints, migraciones y pruebas.
- Podrías implementar exactamente los estados del proceso.
- Sería más fácil representar releases agrupados.
- Podrías automatizar el cálculo de Health.
- La integración con Jira sería controlable.
- Podrías agregar nuevos proyectos sin crear nuevas páginas o archivos.
- No dependerías de conocer Power Apps o Power Automate en profundidad.

Riesgos

1. Convertirse en una aplicación personal

Si solamente tú conoces y mantienes el sistema, existe riesgo operativo.

La aplicación debería tener:

- Repositorio corporativo.
- Documentación.
- Revisiones de código.
- Tests.
- Pipeline.
- Copropietario técnico.
- Backups.
- Monitoreo.
- Responsable de soporte.

2. Hosting

Antes de desarrollar debemos saber:

- ¿Dónde se ejecutará Angular?
- ¿Dónde se ejecutará la API?
- ¿Dónde estará MySQL?
- ¿Existe un ambiente de desarrollo?
- ¿Quién despliega?
- ¿Quién administra los certificados y dominios?
- ¿Quién monitorea la aplicación?

3. Seguridad

El sistema tendría nombres, proyectos, estados y posiblemente información de producción. Debe acordarse:

- Qué información está permitida.
- Quién puede verla.
- Qué acciones puede ejecutar cada rol.
- Cómo se almacenan los secretos.
- Cuánto tiempo se conserva el historial.
- Cómo se realizan los backups.

4. Codex no elimina la validación

Codex puede acelerar el desarrollo, pero el código todavía necesita:

- Revisión.
- Pruebas unitarias.
- Pruebas de integración.
- Análisis de seguridad.
- Validación funcional.
- Gestión de dependencias.
- Protección de secretos.
- Pipeline y deployment controlado.

MVP recomendado

Implementaría inicialmente:

1. Autenticación.
2. Usuarios y roles.
3. Proyectos.
4. Módulos.
5. Release Register.
6. Estados por ambiente.
7. Producción y activación.
8. Follow-ups.
9. Historial.
10. Dashboard básico.
11. Filtros.
12. Enlaces a Jira.
13. Exportación a Excel.

Dejaría para una fase posterior:

- Sincronización automática con Jira.
- Teams.
- Correo automático.
- Power BI.
- Aprobaciones complejas.
- Importación masiva.
- Aplicación móvil.

- Predicciones.
- Ejecución de deployments.

Recomendación final

El desarrollo con Angular, NestJS y MySQL puede ser una mejor alternativa para ti que Power Platform, siempre que la organización pueda proporcionar:

- Hosting.
- Autenticación corporativa.
- Repositorio.
- Seguridad.
- Code review.
- Deployment.
- Soporte y mantenimiento.

No recomendaría construirlo como una herramienta personal aislada. Lo presentaría como una iniciativa interna con un PRD, un MVP y ownership compartido.

La opción técnicamente más equilibrada sería:

- Angular + NestJS + MySQL para la aplicación.
- Microsoft Entra ID para acceso.
- Jira API inicialmente en modo lectura.
- Power Automate o Teams solamente para notificaciones.
- Reports como piloto.
- Expansión posterior a AdminAPI, Dashboard, Invoicing y Fintrac.

El siguiente paso debería ser elaborar el PRD de forma neutral, comparar formalmente Power Platform contra desarrollo propio y definir una arquitectura del MVP antes de escribir código.

2026-08-14 18:16:13 **You**

he trabajado con Laravel, lo tengo instalado en mi maquina. Por el momento sería una aplicación local

ChatGPT gpt-5.6-sol-wm

Eso reduce bastante el costo y el riesgo del primer prototipo. Voy a ajustar la evaluación a un MVP local con Laravel, MySQL y Angular, distinguiendo claramente qué podemos simplificar ahora y qué conviene diseñar desde el inicio para que después pueda convertirse en una aplicación compartida. No iniciaré desarrollo todavía.

Eso hace que Laravel + Angular + MySQL sea una alternativa todavía más apropiada para el MVP. Al ser inicialmente una aplicación local, podemos reducir considerablemente el alcance técnico y concentrarnos en validar el proceso de control.

Mi recomendación sería tratarla como un prototipo funcional personal, no todavía como el sistema oficial del equipo.

Arquitectura local

```text
Angular
   ↓ HTTP/JSON
Laravel REST API
   ↓
MySQL
```

Ejecución inicial:

- Angular: interfaz local.
- Laravel: API y reglas de negocio.
- MySQL: base de datos local.
- Archivos: almacenamiento local de evidencias.
- Usuario: Jhon.
- Acceso: únicamente desde tu equipo.

Dos alternativas de implementación

Angular + Laravel API

Es mi recomendación si existe una probabilidad real de convertirla posteriormente en una aplicación compartida.

Ventajas:

- Aprovecha tu experiencia en Angular.
- Mantiene separadas la interfaz y las reglas de negocio.
- Facilita una futura integración con Jira.
- Permite reemplazar la autenticación local por Microsoft Entra ID.

- El Backend puede utilizarse posteriormente desde otras interfaces.
- Escala mejor hacia una solución multiusuario.

Desventaja:

- Requiere mantener dos proyectos.
- Necesita configurar CORS, autenticación y comunicación SPA/API.
- El desarrollo inicial será un poco mayor.

Laravel con Blade o Livewire

Sería más rápido para un prototipo puramente local:

- Un solo proyecto.
- Formularios y tablas más rápidos de implementar.
- Autenticación y sesiones integradas.
- Menor configuración.

Sin embargo, si después quieres Angular, parte de la interfaz tendría que rehacerse. Como ya tienes experiencia con Angular y el sistema puede evolucionar, mantendría Angular + Laravel.

Qué podemos simplificar en la versión local

Autenticación

Podríamos comenzar con un único usuario local o incluso sin pantalla de acceso.

No obstante, dejaría desde el principio:

- Tabla `users`.
- Campo `created_by`.
- Campo `updated_by`.
- Roles.
- Registro de auditoría.

Así no tendremos que cambiar el modelo cuando se convierta en multiusuario.

Si deseas implementar autenticación desde el MVP, Laravel Sanctum está diseñado para SPAs que consumen una API Laravel mediante sesiones y protección CSRF. Angular HttpClient puede trabajar con el token CSRF generado por Laravel. Laravel: Sanctum SPA authentication

Notificaciones

En la primera versión no enviaría correos ni mensajes de Teams.

La aplicación mostraría:

- Acciones vencidas.

- Próximos seguimientos.
- Registros sin actualización.
- Producción sin activación.
- Bloqueos.

Más adelante se podrían agregar notificaciones externas.

Jira

Inicialmente almacenaría:

- Jira Key.
- URL.
- Título opcional.
- Comentarios.

No implementaría sincronización automática durante el MVP.

Evidencias

Podríamos comenzar con:

- Texto de evidencia.
- URL.
- Nombre de la persona que confirmó.
- Fecha de confirmación.

Los archivos adjuntos podrían agregarse después.

Limitaciones de una aplicación local

No será colaborativa

Ximena, QA o Ameet no podrán actualizar directamente la información.

Tú continuarás recopilando las confirmaciones y registrándolas.

No estará disponible cuando tu máquina esté apagada

Esto afecta especialmente:

- Recordatorios.
- Procesos programados.
- Sincronizaciones.
- Notificaciones.
- Backups automáticos.

Laravel tiene un scheduler para ejecutar tareas periódicas, pero necesita que la máquina y el proceso correspondiente estén activos. Laravel: Task scheduling

Durante el MVP, los recordatorios podrían calcularse al abrir la aplicación, sin depender de un proceso ejecutándose permanentemente.

No debería almacenar información sensible

El piloto debería contener únicamente:

- Estados.
- Nombres de proyectos.
- Tickets.
- Fechas.
- Responsables.
- Evidencia no sensible.

No incluiría:

- Datos de clientes.
- Credenciales.
- Información de producción protegida.
- Tokens de Jira dentro del código.
- Archivos confidenciales.

Necesita backup

Como mínimo:

- Repositorio Git privado para el código.
- Archivo `.env` excluido del repositorio.
- Exportación periódica de MySQL.
- Copia del directorio de evidencias.
- Migraciones para reconstruir la estructura.
- Seeders para datos de configuración.

MVP local recomendado

1. Portfolio Dashboard

- Reports.
- AdminAPI.
- Dashboard.

- Invoicing.
- Fintrac.
- Salud general.
- Releases activos.
- Acciones vencidas.
- Bloqueos.
- Pendientes de producción.

2. Projects

- Proyecto.
- Descripción.
- Tech Lead.
- Stakeholders.
- Prioridad.
- Estado.
- Última revisión.

3. Modules

- Proyecto.
- Producto o módulo.
- Descripción.
- Responsable.
- Estado.

4. Releases

- Release ID.
- Proyecto.
- Nombre.
- Alcance.
- Ambiente objetivo.
- Fecha planeada.
- Estado.
- Release notes.

5. Release Items

- Producto o módulo.
- Alcance.
- Jira tickets.
- MOJA.
- TATU.
- SITA.
- Producción.
- Activación.
- Próxima acción.
- Responsable.
- Fecha de seguimiento.
- Evidencia.

6. Follow-ups

- Acción.
- Proyecto.
- Release.
- Responsable del seguimiento.
- Contacto.
- Fecha.
- Estado.
- Resultado.

7. History

- Entidad modificada.
- Acción.
- Valor anterior.
- Valor nuevo.
- Fecha.
- Usuario.
- Evidencia.

8. Configuration

- Ambientes.
- Estados.
- Prioridades.
- Tipos de evento.
- Estados de activación.
- Criterios de salud.

Reglas que deben implementarse desde el principio

- Los ambientes deben ser configurables.
- “Production Deployment” y “Client Activation” deben estar separados.
- Un release puede contener varios release items.
- Un release item puede estar en un ambiente diferente de los demás.
- Todo estado debe tener fecha y fuente.
- Todo elemento abierto debe tener una próxima acción.
- Todo seguimiento debe tener responsable y fecha.
- Los cambios de estado deben generar historial.
- Los registros no deberían eliminarse físicamente sin una política definida.
- Remittances no debe formar parte de los proyectos actuales del sistema.

Automatizaciones locales iniciales

No necesitan procesos externos. Pueden calcularse cuando abras el dashboard:

Health

- Blocked: existe un bloqueo activo.
- Overdue: la fecha de seguimiento venció.
- Needs Follow-up: producción o activación está sin confirmar.
- In Progress: desarrollo, merge o pruebas en curso.
- Action Required: existe próxima acción pendiente.
- On Track: no existen excepciones.

Alertas

- Follow-up date menor que hoy.
- Last verified mayor que determinado número de días.

- Producción desplegada con activación sin confirmar.
- Release item sin próxima acción.
- Release sin release notes.
- Elemento aprobado sin promoción programada.

Preparación para una futura publicación

Aunque sea local, diseñaría el sistema para que posteriormente pueda publicarse:

- API REST independiente.
- Variables de entorno.
- Migraciones.
- Seeders.
- Roles y permisos.
- Auditoría.
- IDs estables.
- Fechas en UTC.
- Almacenamiento abstraído.
- Servicios de notificación separados.
- Integración con Jira encapsulada.
- Pruebas unitarias e integración.
- Documentación de instalación.

Así, el paso de local a compartido requeriría principalmente:

- Hosting.
- Dominio.
- HTTPS.
- Base de datos administrada.
- Autenticación corporativa.
- Backups.
- Monitoreo.
- Pipeline.
- Revisión de seguridad.

Recomendación final

Para tu situación, desarrollaría el MVP local con:

- Angular para Frontend.
- Laravel como REST API.
- MySQL.
- Sin integración automática con Jira.
- Sin Power BI.
- Sin Teams.
- Sin notificaciones externas.
- Sin información sensible.
- Con Reports como primer conjunto de datos.
- Con soporte estructural para los otros cuatro proyectos.

El Excel que creamos serviría para:

- Poblar los datos iniciales.
- Validar las pantallas.
- Definir los campos.
- Comparar resultados.
- Preparar los casos de prueba.

Con esta aclaración, sí considero razonable avanzar con un PRD orientado a un MVP local, dejando explícitamente fuera del primer alcance el hosting, la colaboración multiusuario y las integraciones corporativas.

2026-08-14 18:22:06 **You**

Ayudame con un documento PRD orientado a un MVP local

ChatGPT gpt-5.6-sol-wm

Voy a crear el PRD como documento formal en español, orientado específicamente al MVP local con Angular, Laravel y MySQL. Incorporaré Reports como caso inicial, soporte futuro para AdminAPI, Dashboard, Invoicing y Fintrac, y dejaré Remittances explícitamente fuera del alcance. Usaré un formato verificable y listo para revisión.

Estoy aplicando la guía de documentos para que el PRD no sea solo un borrador textual: tendrá requisitos trazables, criterios de aceptación y una presentación lista para revisión. También lo guardaré como un artefacto reutilizable.

Ya consolidé las decisiones clave: el MVP será local y de un solo usuario, con Reports como piloto; deberá poder ampliarse a AdminAPI, Dashboard, Invoicing y Fintrac, y excluirá explícitamente Remittances. Ahora estoy convirtiendo esas decisiones en requisitos verificables y no solo en una descripción técnica.

El primer render produjo 16 páginas. Estoy revisando una por una para detectar cortes, tablas demasiado densas, saltos incómodos y cualquier diferencia entre el contenido y la maquetación.

La revisión visual detectó que varias secciones extensas compartían página y comprimían el encabezado o pie. Estoy separando esas secciones por bloques de lectura —requisitos, baseline, validación, riesgos y aprobaciones— para que el PRD quede estable en Word y no dependa del motor de paginación.

El PRD ya pasó la revisión visual completa de 22 páginas: tablas, encabezados, pies, saltos y numeración están estables. La matriz original quedó integrada como baseline de Reports, no como único modelo de control. Ahora estoy guardando la versión final para que puedas reutilizarla y versionarla.

He creado el PRD completo para el MVP local. Incluye alcance, requisitos funcionales y no funcionales, arquitectura Angular–Laravel–MySQL, modelo de datos, reglas de negocio, baseline de Reports, criterios de aceptación, UAT, riesgos y preguntas abiertas.

Remittances está explícitamente excluido y se contempla la futura incorporación de AdminAPI, Dashboard, Invoicing y Fintrac.

Descargar PRD — MVP local

También quedó guardado en tu Library, dentro de la carpeta `REPORTS`.

2026-08-14 18:58:44 **You**

Traducelo literal a Inglés. NO menciones Remittances. Utiliza PROD para referirte al ambiente de producción. Los ambientes son MOJA, TATU, SITA, PROD. Ademas prefiero un archivo Markdown

ChatGPT gpt-5.6-sol-wm

Usaré las instrucciones de documentos para preservar la estructura del PRD y las de Library porque crearé un nuevo archivo reutilizable. Traduciré el contenido al inglés de forma literal, eliminaré toda mención a Remittances y normalizaré los ambientes como MOJA, TATU, SITA y PROD.

Ya recuperé la versión completa del PRD. Mantendré la misma numeración, requisitos, criterios, tablas y decisiones; solo aplicaré las tres modificaciones solicitadas: traducción literal al inglés, supresión

total del tema indicado y uso consistente de `PROD`.
La traducción ya está terminada y pasó las comprobaciones de contenido: no contiene la referencia excluida y emplea `PROD` de forma uniforme. Ahora haré la validación final del Markdown y lo guardaré como un archivo nuevo en Library, conservando intacto el PRD original en español.

He creado la traducción literal al inglés en formato Markdown, usando exclusivamente los ambientes MOJA, TATU, SITA y PROD. El tema solicitado fue omitido completamente.

Descargar PRD en inglés — Markdown

También quedó guardado en Library, dentro de . El documento original en español permanece intacto.

2026-08-15 15:02:29 **You**

Me puedes ayudar a crear una estructura de base de datos apropiada para la implementación del sistema?

ChatGPT gpt-5.6-sol-wm

Sí. Voy a convertir el modelo conceptual del PRD en un diseño relacional implementable con Laravel y MySQL. Prepararé una especificación Markdown con tablas, relaciones, restricciones, índices, estados iniciales y un diagrama ER; usaré `PROD` de forma consistente y mantendré el alcance del MVP local. También la guardaré como un artefacto separado en Library.

El diseño ya está estructurado. Incluí una decisión importante para evitar inconsistencias: una release pertenece al proyecto y puede contener ítems de distintos módulos, pero llaves foráneas compuestas impedirán que un ítem se vincule accidentalmente con un módulo, ambiente o release de otro proyecto. Estoy validando ahora las tablas, relaciones e índices.

La estructura pasó la validación del Markdown y de consistencia de sus 20 tablas documentales. La guardaré en Library dentro de `/REPORTS` como una especificación técnica independiente del PRD.

Preparé el diseño de base de datos para el MVP local con Laravel y MySQL.

Incluye:

- 14 tablas operativas y de configuración.
- Diagrama ER en Mermaid.
- Diccionario completo de campos, tipos y restricciones.
- Llaves foráneas e índices recomendados.
- Estados iniciales y seeders.

- Separación entre despliegue en PROD y activación para clientes.
- Historial append-only y evidencias.
- Reglas para calcular la salud sin almacenarla.
- Orden recomendado de Laravel migrations.
- Mapeo de la matriz actual al nuevo modelo.
- Implementación dividida en cuatro incrementos.

Decisiones principales:

- ULID para entidades operativas.
- Una release puede incluir ítems de diferentes módulos del mismo proyecto.
- MOJA, TATU, SITA y PROD son ambientes globales, configurables y ordenables por proyecto.
- Las FK compuestas evitan relaciones entre proyectos incorrectos.
- Laravel migrations será la fuente de verdad del esquema.

Descargar diseño de base de datos — Markdown

También quedó guardado en Library dentro de `/REPORTS`.

El siguiente paso recomendado sería convertir este diseño en las migraciones, modelos Eloquent, relaciones y seeders iniciales de Laravel.